```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.MonsterInn.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S99_Boot";
        public const string Main = "S01_InnHall";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.MonsterInn.Boot
{
    /// &lt;summary&gt;
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
    static bool ConfigStep()
    {
```

```
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.MonsterInn.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
    }
    public sealed class MockTimeService : ITimeService
    {
```

```
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
    public void Preload(string pointId)
    {
        Logger.Info("ADS", "preload (mock): " + pointId);
```

```
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// AdFlow.cs
// AdFlow.cs - S1 rewarded-ad capsule core (T008): ports the frozen E4 graybox M11
// face into the real runtime. Three ad slots, all rewarded, all purely optional
// (U090-3 heal law: zero interstitials, zero forced views, zero interruptions of
// the cozy night loop - an offer is a quiet button the player may tap, never a
// wall the game throws up). The 12-per-day hard cap (U149 dual-zone: the same cap
// binds the domestic and the overseas compliance zone - switching zones never
// loosens it) rides a rolling 24h window in the WALL-CLOCK domain (X995 law: an
// uptime-domain window dies across sessions - the new session clock restarts near
// zero, old capsules look future-dated, the prune condition never fires and the
// cap silently becomes a lifetime meter, compliant but UX-dead; wall seconds keep
// the window truly rolling across restarts). OfferAd prunes-then-caps-then-records;
// a capped offer is a silent refusal with zero mutation outside the ad ledger and
// the night keeps flowing untouched (E4 assertion-6: 20 offers / 0 accepted and the
// full settlement loop still closes - no system in this inn is ever blocked by an
// ad). Accepting is strictly post-offer and once-per-capsule: the boost grant
// itself lives at the consuming system's call site (serve double / rare summon /
// stay-claim double bind in T009), so the ad machine never touches the wallet, the
// codex or the shift. Pure C# (no UnityEngine) so Editor gates drive it headless.
// Every number is a frozen E4 anchor (GrayboxSim.cs) - zero re-tuning, zero RNG.
using System;
using System.Collections.Generic;
using System.Globalization;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;
    /// Frozen ad face (E4 GrayboxSim AdsRequiredForClosure + admission S9): the three
    /// rewarded categories, the U149 dual-zone daily cap and the X995 rolling window.
    /// Zero re-tuning: S2+ content work widens flavor text, never the cap or window.
    /// &lt;/summary&gt;
    public static class AdFrozen
    {
        public const int DailyCap = 12;                // U149: 12 offers per rolling 24h, both zones
        public const double WindowSeconds = 86400.0;   // X995: 24h rolling window, wall-clock domain
        public const bool AdsRequiredForClosure = false; // E3#4/E4: ads purely optional
        public const bool InterstitialAllowed = false;   // U090-3: rewarded only, zero interstitials
        public static readonly string[] Categories =
        {
            "AD_SERVICE_DOUBLE",    // rewarded, optional: double tonight's serving earnings
            "AD_RARE_SUMMON",      // rewarded, optional: summon a rare yokai guest tonight
            "AD_OFFLINE_CLAIM"     // rewarded, optional: double tonight's stay earnings
        };
        // Frozen offer captions (graybox BoostOffer voice, per-category). The refused
        // face stays cozy: a capped offer is a quiet no, never a nag.
        public static readonly string[] OfferDetails =
        {
```

```
        "optional: watch a short video to double tonight's serving earnings?",
        "optional: watch a short video to summon a rare yokai guest tonight?",
        "optional: watch a short video to double tonight's stay earnings?"
    };
    public static bool IsCategory(string type)
    {
        for (int i = 0; i &lt; Categories.Length; i++) if (Categories[i] == type) return true;
        return false;
    }
}
/// &lt;summary&gt;One ad slot exposure (S9 data face): category + wall stamp + accept flag.&lt;/summary&gt;
public sealed class AdCapsule
{
    public string Type;        // one of AdFrozen.Categories
    public double OfferedAt;   // wall-clock seconds at offer time (X995 domain)
    public bool Accepted;      // one accept per capsule (strictly post-offer opt-in)
}
/// &lt;summary&gt;
/// Ad ledger (S9 data face, save.adcaps). The capsule list is the rolling-window
/// truth; lifetime counters ride along for the IAA-health metrics face.
/// &lt;/summary&gt;
public sealed class AdLedger
{
    public readonly List&lt;AdCapsule&gt; Capsules = new List&lt;AdCapsule&gt;();
    public int OffersTotal;     // lifetime offers that passed the cap
    public int AcceptsTotal;    // lifetime accepts
}
/// &lt;summary&gt;
/// M11 rewarded-ad capsule machine (S9 API face). The caller (view layer, T009)
/// stamps the wall clock at offer time; the machine owns prune / cap / record /
/// accept discipline. It never self-fires and never calls into the game systems -
/// the zero-blocking law is structural, not just behavioral.
/// &lt;/summary&gt;
public sealed class AdFlow
{
    public double Time;   // logical seconds (event stamps; view binds in T009)
    public readonly List&lt;InnEvent&gt; Events = new List&lt;InnEvent&gt;();
    public readonly AdLedger Ledger = new AdLedger();
    public void Log(string type, string detail) { Events.Add(new InnEvent(Time, type, detail)); }
    public double TimeOf(string type)
    {
        for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) return Events[i].T;
        return -1.0;
    }
    public int CountEvents(string type)
    {
        int n = 0;
        for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) n++;
        return n;
    }
```

```
// X995 rolling-window prune: a capsule ages out at exactly 24h (inclusive
// boundary). A rolled-back wall clock never prunes - a future-dated capsule
// stays counted, so the machine can only under-offer, never over-offer.
public int Prune(double wallNow)
{
    int removed = 0;
    for (int i = Ledger.Capsules.Count - 1; i &gt;= 0; i--)
    {
        if (wallNow - Ledger.Capsules[i].OfferedAt &gt;= AdFrozen.WindowSeconds)
        {
            Ledger.Capsules.RemoveAt(i);
            removed++;
        }
    }
    return removed;
}
// In-window offers of one category (post-prune face; the view may hide a slot
// whose category already saturated the window).
public int CountInWindow(string type, double wallNow)
{
    int n = 0;
    for (int i = 0; i &lt; Ledger.Capsules.Count; i++)
    {
        if (Ledger.Capsules[i].Type == type
            &amp;&amp; wallNow - Ledger.Capsules[i].OfferedAt &lt; AdFrozen.WindowSeconds) n++;
    }
    return n;
}
// M11 offer face (S9 API): prune -&gt; cap -&gt; record. Returns true when the slot
// exposure is allowed (a capsule lands in the rolling window), false when the
// U149 cap or an unknown category refuses it. Refusal is a silent no-op: zero
// mutation outside this ledger, zero events on the game's core streams, and
// the caller simply does not show the button - the night flows on untouched.
public bool OfferAd(string type, double wallNow)
{
    Prune(wallNow);
    if (!AdFrozen.IsCategory(type)) return false;          // zero interstitials, zero stray types
    if (Ledger.Capsules.Count &gt;= AdFrozen.DailyCap)
    {
        Log("ad_cap_refused", "daily cap reached: the inn hums along untouched");
        return false;
    }
    Ledger.Capsules.Add(new AdCapsule { Type = type, OfferedAt = wallNow });
    Ledger.OffersTotal++;
    int cat = Array.IndexOf(AdFrozen.Categories, type);
    Log("ad_offer", AdFrozen.OfferDetails[cat]);
    return true;
}
// M11 accept face: strictly post-offer opt-in, once per capsule. No open,
// un-accepted, in-window capsule of this type = zero accepts, zero mutation
```

```
        // (an empty ledger, a capped-out face, an aged-out capsule or a second tap on
        // the same capsule all refuse). The boost grant lives at the consuming call
        // site - this face only writes the accept bookkeeping.
        public bool AcceptOffer(string type, double wallNow)
        {
            if (!AdFrozen.IsCategory(type)) return false;
            AdCapsule best = null;
            for (int i = 0; i &lt; Ledger.Capsules.Count; i++)
            {
                AdCapsule c = Ledger.Capsules[i];
                if (c.Type != type || c.Accepted) continue;
                if (wallNow - c.OfferedAt &gt;= AdFrozen.WindowSeconds) continue;  // aged out: gone
                if (best == null || c.OfferedAt &gt; best.OfferedAt) best = c;      // newest open slot
            }
            if (best == null) return false;
            best.Accepted = true;
            Ledger.AcceptsTotal++;
            Log("ad_accept", type);
            return true;
        }
    }
}
// CodexFlow.cs
// CodexFlow.cs - S1 codex + upgrade core (T005): ports the frozen E4 graybox M6/M7
// face into the real runtime. Checkout-driven first-seen unlock: UnlockEntry is
// idempotent - a repeat species is a warm reunion, never a second entry, never a
// reset, so the 12-slot codex wall is pure collection with zero punishment (E3#5).
// The inn upgrade spends the level's curve cost (4/8/14/22/32/44, T012-c) for +1
// cozy room; InnLevel grows by one on every success and never goes down (the
// growth spine). A refused upgrade - empty wallet, below cost, or the full
// build-out at level 7 - is honest scarcity, not punishment: zero wallet
// mutation, zero events.
// The wallet is injected per call (ServeShift.Wallet is the single economy source)
// so codex and shift can never drift into two coin ledgers. Pure C# (no UnityEngine)
// so Editor gates drive it headless; the view layer binds in T009. Every number is a
// frozen E4 anchor (GrayboxSim.cs) - zero re-tuning, zero RNG by design.
using System;
using System.Collections.Generic;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;One yokai codex wall slot (S4 data face, save.codex).&lt;/summary&gt;
    public sealed class CodexEntry
    {
        public string Species;   // frozen roster token
        public string Rarity;    // S2 tier data slot (common / rare / epic, T012-b)
        public double UnlockT;   // night clock of the first-seen checkout
    }
    /// &lt;summary&gt;Inn growth state (S5 data face, save.inn).&lt;/summary&gt;
    public sealed class InnState
    {
```

```
    public int Level = 1;                        // monotonically increasing spine
    public double Cost = InnFrozen.UpgradeCost;  // next-step curve cost face (save mirror, T012-c)
}
/// &lt;summary&gt;
/// M6 codex unlocks + M7 inn upgrade machine. The caller pumps Tick(dt) on the
/// shared night clock (kept in lockstep with the ServeShift clock by the bot);
/// UnlockEntry fires on the checkout beat and UpgradeInn on the player tap.
/// &lt;/summary&gt;
public sealed class CodexFlow
{
    public double Time;   // night-local logical seconds
    public readonly List&lt;InnEvent&gt; Events = new List&lt;InnEvent&gt;();
    public readonly List&lt;CodexEntry&gt; Entries = new List&lt;CodexEntry&gt;();
    public readonly InnState Inn = new InnState();
    public int CozyRooms;
    public int CountUnlocked { get { return Entries.Count; } }
    public void Log(string type, string detail) { Events.Add(new InnEvent(Time, type, detail)); }
    public double TimeOf(string type)
    {
        for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) return Events[i].T;
        return -1.0;
    }
    public int CountEvents(string type)
    {
        int n = 0;
        for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) n++;
        return n;
    }
    // Advance the night clock (unlock/upgrade beats are caller-driven, not self-firing).
    public void Tick(double dt) { Time += dt; }
    public bool HasSpecies(string species)
    {
        for (int i = 0; i &lt; Entries.Count; i++) if (Entries[i].Species == species) return true;
        return false;
    }
    // M6: first-seen unlock (S4 API face). Idempotent by design and frozen to the
    // 12-species roster: non-roster tokens can never open a 13th slot.
    public bool UnlockEntry(string species)
    {
        if (!IsRosterSpecies(species) || HasSpecies(species)) return false;
        Entries.Add(new CodexEntry { Species = species, Rarity = InnFrozen.RarityOfSpecies(species), UnlockT = Time
        Log("codex_unlock", species + " joins your yokai codex");
        InnAudioHooks.OnCodexUnlock();  // T010: same-tick unlock chime hook
        return true;
    }
    // M7: the inn upgrade (S5 API face). Spends the level's curve cost from the
    // shared wallet, adds one cozy room and grows the level by one. The full
    // build-out (level 7) and any below-cost tap are refused silently with zero
    // mutation - scarcity is never a punishment.
    public bool UpgradeInn(FortuneWallet wallet)
```

```
        {
            if (Inn.Level &gt;= InnFrozen.InnMaxLevel) return false;
            if (wallet == null || !wallet.SpendCoins(InnFrozen.UpgradeCostFor(Inn.Level))) return false;
            Inn.Level++;
            Inn.Cost = InnFrozen.UpgradeCostFor(Inn.Level);
            CozyRooms++;
            Log("inn_upgrade", "inn level " + Inn.Level + ": one more cozy room");
            InnAudioHooks.OnUpgrade();  // T010: same-tick upgrade swell hook
            return true;
        }
        private static bool IsRosterSpecies(string species)
        {
            for (int i = 0; i &lt; InnFrozen.CodexRoster.Length; i++)
                if (InnFrozen.CodexRoster[i] == species) return true;
            return false;
        }
    }
}
// EventFlow.cs
// EventFlow.cs - comedy event-flow core (T007 S1 face, T012-d S2 curve): ports the
// frozen E4 graybox M9 face into the real runtime. The S2 frequency curve F(n)
// (B712 numeric freeze) serves 1 comedy event per night for nights 0-9 (calm
// onboarding) and 2 per night for nights 10+ (retention escalation): three
// families rotate deterministically by cumulative event index (zero RNG), so a
// 20-night run lands the frozen 10/10/10 split with every family seen at least
// once. Within a family the text keys walk the frozen text bank in order
// (event_news_1-10 / event_docu_1-10 / event_wuxia_1-10), so the first 20 nights
// consume each bank exactly once (stylistic-signature law: each family keeps its
// own voice). The event ledger (S7 data face, save.events) carries
// the per-family counts and the last fired night; a night never fires twice - a
// repeat or older night index is a silent no-op, and that per-night key doubles as
// the offline catch-up cap (replaying missed nights after a resume can never stack
// duplicate events). Pure C# (no UnityEngine) so Editor gates drive it headless;
// the view layer binds in T009. Every number is a frozen E4 anchor (GrayboxSim.cs)
// - zero re-tuning, zero RNG by design.
using System;
using System.Collections.Generic;
using System.Globalization;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;
    /// Frozen three-family comedy face (E4 GrayboxSim.ComedyEventFamilies + the
    /// 20-night 10/10/10 split): family names, log event types, text-key prefixes
    /// and per-family text bank sizes, plus the S2 frequency curve F(n) (B712
    /// freeze). Zero re-tuning: later content batches widen the banks, they never
    /// re-shape the rotation.
    /// &lt;/summary&gt;
    public static class ComedyFace
    {
        public const int FamilyCount = 3;
```

```
    public static readonly string[] Families =
    {
        "yao_news_broadcast",      // yokai nightly news anchor style
        "documentary_monologue",   // "he waited one hundred years for hot water" style
        "wuxia_guest_arrival"     // wuxia-style swordsman yokai checks in
    };
    public static readonly string[] FamilyEventTypes =
    {
        "comedy_news", "comedy_docu", "comedy_wuxia"
    };
    public static readonly string[] FamilyKeyPrefixes =
    {
        "event_news_", "event_docu_", "event_wuxia_"
    };
    // S2 text-bank sizes (T012-d content batch): ten per family - the F(n)
    // 1-&gt;2 curve consumes exactly 10/10/10 across the first 20 nights.
    public static readonly int[] FamilyTextCounts = { 10, 10, 10 };
    // S2 frequency curve F(n) (B712 numeric freeze): calm onboarding 1/night
    // for nights 0-9, retention escalation 2/night for nights 10+. Family is
    // a pure function of the cumulative event index (replay-safe, zero RNG).
    public static int ComedyEventsForNight(int nightIndex) { return nightIndex &lt; 10 ? 1 : 2; }
    public static int ComedyEventsBefore(int nightIndex)
    {
        int total = 0;
        for (int n = 0; n &lt; nightIndex; n++) total += ComedyEventsForNight(n);
        return total;
    }
    public static int ComedyFamilyOf(int nightIndex, int k)
    {
        return (ComedyEventsBefore(nightIndex) + k) % FamilyCount;
    }
    // Text key for the k-th fire of a family (occurrence is 0-based; keys are
    // 1-based per the i18n key list). Walks the bank in order and wraps.
    public static string TextKeyOf(int family, int occurrence)
    {
        int slot = occurrence % FamilyTextCounts[family];
        return FamilyKeyPrefixes[family]
            + (slot + 1).ToString(CultureInfo.InvariantCulture);
    }
}
/// &lt;summary&gt;Comedy event ledger (S7 data face, save.events).&lt;/summary&gt;
public sealed class EventLedger
{
    public readonly int[] FamilyCount = new int[ComedyFace.FamilyCount]; // lifetime fires per family
    public int LastEventNight = -1;   // idempotency key: one comedy beat per night
    public int TotalFired;            // lifetime total
}
/// &lt;summary&gt;
/// M9 nightly comedy event machine (S7 API face). The caller fires NightComedy
/// on the night beat; family rotation, in-bank text walk and the per-night
```

```
    /// idempotency all live here (the product owns its rules, not the caller).
    /// &lt;/summary&gt;
    public sealed class EventFlow
    {
        public double Time;   // night-clock logical seconds (view binds in T009)
        public readonly List&lt;InnEvent&gt; Events = new List&lt;InnEvent&gt;();
        public readonly EventLedger Ledger = new EventLedger();
        public void Log(string type, string detail) { Events.Add(new InnEvent(Time, type, detail)); }
        public double TimeOf(string type)
        {
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) return Events[i].T;
            return -1.0;
        }
        public int CountEvents(string type)
        {
            int n = 0;
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) n++;
            return n;
        }
        // Advance the night clock (the comedy beat is caller-driven, not self-firing).
        public void Tick(double dt) { Time += dt; }
        // M9: the nightly comedy beat. S2 frequency curve F(n) (T012-d): nights 0-9
        // fire one event, nights 10+ fire two - family rotation, in-bank text walk
        // and the per-night idempotency all live here (the product owns its rules,
        // not the caller). A repeat or older night index is a silent no-op with
        // zero mutation - the per-night key is also the offline catch-up cap (no
        // stacked replays).
        public bool NightComedy(int nightIndex)
        {
            if (nightIndex &lt;= Ledger.LastEventNight) return false;
            int fires = ComedyFace.ComedyEventsForNight(nightIndex);
            for (int k = 0; k &lt; fires; k++)
            {
                int fam = ComedyFace.ComedyFamilyOf(nightIndex, k);
                string textKey = ComedyFace.TextKeyOf(fam, Ledger.FamilyCount[fam]);
                Ledger.FamilyCount[fam]++;
                Ledger.TotalFired++;
                Log(ComedyFace.FamilyEventTypes[fam], textKey);
                InnAudioHooks.OnEventComedy();  // T010: same-tick comedy event sting hook
            }
            Ledger.LastEventNight = nightIndex;
            return true;
        }
    }
}
// InnAudioCatalog.cs
// InnAudioCatalog.cs - T010 generated-audio catalog (X027 three-piece: catalog SO +
// null-safe static lookup). Mirror of the G08 CampusAudioCatalog paradigm: the
// arrival-batch setup tool (section 5.3 wiring contract) scans Assets/_Game/Art/Audio
// and serializes clip references here so runtime code resolves audio by filename stem
```

```
// (InnAudioStems). The asset lives at Assets/_Game/Resources/InnAudioCatalog.asset.
// Missing clip / missing catalog asset = null = stay silent (X027 null-safe law):
// headless gates stay green in the pre-arrival skeleton state, and the audio wave
// landing later lights the whole face up with zero code churn.
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.MonsterInn
{
    [CreateAssetMenu(fileName = "InnAudioCatalog", menuName = "Sy/MonsterInn Audio Catalog")]
    public sealed class InnAudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;      // filename stem, e.g. "g20_bgm_inn_night"
            public AudioClip clip; // direct reference keeps builds self-contained
        }
        public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
        Dictionary&lt;string, AudioClip&gt; _index;
        void OnEnable()
        {
            // Serializer revival can pair a fresh instance with a stale cache;
            // drop it so the next Get() rebuilds from entries.
            _index = null;
        }
        public AudioClip Get(string key)
        {
            if (string.IsNullOrEmpty(key)) return null;
            if (_index == null)
            {
                _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
                for (int i = 0; i &lt; entries.Count; i++)
                {
                    var e = entries[i];
                    if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                        _index[e.key] = e.clip;
                }
            }
            AudioClip c;
            return _index.TryGetValue(key, out c) ? c : null;
        }
    }
    /// &lt;summary&gt;
    /// Runtime lookup helper (CampusAudio mirror). Consumers MUST treat null as
    /// "audio missing -&gt; stay silent" so headless checks stay green on any project
    /// state (catalog absent, clip pruned, name typo). Stem constants are the single
    /// consumption contract both sides code against - they match the section-5.1
    /// work-order ids one-to-one.
    /// &lt;/summary&gt;
```

```
public static class InnAudio
{
    static InnAudioCatalog _catalog;
    public static InnAudioCatalog Catalog
    {
        get
        {
            if (_catalog == null) _catalog = Resources.Load&lt;InnAudioCatalog&gt;("InnAudioCatalog");
            return _catalog;
        }
    }
    public static AudioClip Get(string key)
    {
        var c = Catalog;
        return c != null ? c.Get(key) : null;
    }
    /// &lt;summary&gt;
    /// Loop the requested stem on a fresh host object and return the source
    /// (the BGM bed wiring slot, X1026 smpl-1024 law applies to the asset
    /// itself at the arrival batch, not here). Missing clip resolves to
    /// silence (returns null) - never an error.
    /// &lt;/summary&gt;
    public static AudioSource PlayLoop(string key, float volume = 0.6f)
    {
        var clip = Get(key);
        if (clip == null) return null;
        var go = new GameObject("InnAudio(" + key + ")");
        var src = go.AddComponent&lt;AudioSource&gt;();
        src.clip = clip;
        src.loop = true;
        src.volume = volume;
        src.Play();
        return src;
    }
    /// &lt;summary&gt;
    /// Fire a one-shot feedback stem. Missing clip = silence (no object
    /// spawned, no error) so event-driven wiring is safe pre-arrival.
    /// &lt;/summary&gt;
    public static void PlayOneShot(string key, float volume = 0.8f)
    {
        var clip = Get(key);
        if (clip == null) return;
        var go = new GameObject("InnSfx(" + key + ")");
        var src = go.AddComponent&lt;AudioSource&gt;();
        src.clip = clip;
        src.loop = false;
        src.volume = volume;
        src.Play();
        // static class has no UnityEngine.Object inheritance face, the
        // destroy call must be fully qualified (X924 non-MonoBehaviour law).
```

```
            UnityEngine.Object.Destroy(go, clip.length + 0.05f);
        }
    }
}
// InnAudioHooks.cs
// InnAudioHooks.cs - T010 same-tick audio hook hub (pure C# - no UnityEngine, the
// domain layer stays engine-free). The flow success paths fire these hooks on their
// REAL transitions, which guarantees the audio rides the exact tick the domain beat
// lands on - any caller (view tap, guide walker, bot) drives the same path. The
// arrival beats (knock buildup + door reveal), the service beats (seat / serve /
// checkout), the growth beats (codex unlock / inn upgrade), the stay morning
// settle, the zero-punishment walkout and the nightly comedy event all ride this
// hub. The Unity layer registers the Playback delegate (host maps stem -&gt;
// InnAudio.PlayOneShot); a null delegate means silence, so Editor gates can count
// hooks headless without an audio device and the pre-arrival state stays green.
using System;
namespace Sy.MonsterInn
{
    public static class InnAudioHooks
    {
        /// &lt;summary&gt;Stem playback sink - set by InnAudioHost; null = silent (headless-safe).&lt;/summary&gt;
        public static Action&lt;string&gt; Playback;
        public static int SeatCount;    // op1 seat taps (real success path)
        public static int ServeCount;   // op2 serve triples (same-tick feedback #1)
        public static int SettleCount;  // checkout coin pops (same-tick feedback #2)
        public static int KnockCount;   // arrival buildup knocks (double event beat)
        public static int RevealCount;  // door-open reveal stings (first wow)
        public static int CodexCount;   // first-seen codex unlocks (feedback #3)
        public static int UpgradeCount; // inn upgrade lantern swells
        public static int WalkoutCount; // zero-punishment walkout puffs
        public static int MorningCount;  // morning stay-settle jingles
        public static int EventCount;   // nightly comedy event stings
        /// &lt;summary&gt;Gate helper: clears counters and the playback sink.&lt;/summary&gt;
        public static void Reset()
        {
            Playback = null;
            SeatCount = 0;
            ServeCount = 0;
            SettleCount = 0;
            KnockCount = 0;
            RevealCount = 0;
            CodexCount = 0;
            UpgradeCount = 0;
            WalkoutCount = 0;
            MorningCount = 0;
            EventCount = 0;
        }
        static void Fire(string stem)
        {
            var p = Playback;
```

```
            if (p != null) p(stem);
        }
        /// &lt;summary&gt;Seat tap (op1) - fired by ServeShift.Seat() on its success branch.&lt;/summary&gt;
        public static void OnSeat() { SeatCount++; Fire(InnAudioStems.UiTapGeneric); }
        /// &lt;summary&gt;Serve triple (op2) - fired by ServeShift.Serve() on its success branch.&lt;/summary&gt;
        public static void OnServe() { ServeCount++; Fire(InnAudioStems.ServeDish); }
        /// &lt;summary&gt;Checkout - fired by ServeShift.FinishMeal() on the settle branch.&lt;/summary&gt;
        public static void OnSettle() { SettleCount++; Fire(InnAudioStems.CoinFortunePop); }
        /// &lt;summary&gt;Buildup knock - fired by InnFlow.Tick() on the buildup double event.&lt;/summary&gt;
        public static void OnKnock() { KnockCount++; Fire(InnAudioStems.KnockSoft); }
        /// &lt;summary&gt;Reveal sting - fired by InnFlow.Tick() when the door opens.&lt;/summary&gt;
        public static void OnReveal() { RevealCount++; Fire(InnAudioStems.RevealStingCute); }
        /// &lt;summary&gt;Codex unlock chime - fired by CodexFlow.UnlockEntry() on first-seen success.&lt;/summary&gt;
        public static void OnCodexUnlock() { CodexCount++; Fire(InnAudioStems.CodexUnlockChime); }
        /// &lt;summary&gt;Upgrade swell - fired by CodexFlow.UpgradeInn() on a successful spend.&lt;/summary&gt;
        public static void OnUpgrade() { UpgradeCount++; Fire(InnAudioStems.UpgradeLanternSwell); }
        /// &lt;summary&gt;Walkout puff - fired by ServeShift.GuestWalksOut() (honest loss, zero punishment).&lt;/summar
        public static void OnWalkout() { WalkoutCount++; Fire(InnAudioStems.WalkoutSoftPuff); }
        /// &lt;summary&gt;Morning settle jingle - fired by StayFlow.MorningSettlement() when a night settles.&lt;/summa
        public static void OnMorningSettle() { MorningCount++; Fire(InnAudioStems.MorningSettleJingle); }
        /// &lt;summary&gt;Comedy event sting - fired by EventFlow.NightComedy() on a fresh night.&lt;/summary&gt;
        public static void OnEventComedy() { EventCount++; Fire(InnAudioStems.EventStingComedy); }
    }
}
// InnAudioHost.cs
// InnAudioHost.cs - T010 scene-side audio face (X027 host piece): registers the
// same-tick hook playback sink and starts the looping night BGM bed. Null-safe by
// contract (CampusAudio mirror): missing catalog/clip = silence, never an error,
// so the pre-arrival skeleton runs green and the arrival wave needs zero churn.
// Ensured at runtime by InnPageHost on the InnHall page (idempotent).
using UnityEngine;
namespace Sy.MonsterInn
{
    public sealed class InnAudioHost : MonoBehaviour
    {
        void Start()
        {
            // lambda: PlayOneShot carries an optional volume, method-group conversion
            // to Action&lt;string&gt; is illegal (CS0123) - bind the default explicitly.
            InnAudioHooks.Playback = s =&gt; InnAudio.PlayOneShot(s);
            InnAudio.PlayLoop(InnAudioStems.BgmInnNight);
        }
    }
}
// InnAudioStems.cs
// InnAudioStems.cs - T010 frozen audio stem contract: the eleven section-5.1
// work-order ids one-to-one (g20-prefixed). The catalog, the same-tick hooks and
// the AudioCheck all code against these constants, so the arrival wave (TJ custom
// for the signature stems, AA candidates for the generic trio) must land files
// under Assets/_Game/Art/Audio with exactly these filename stems - zero drift
```

```
// between the work orders, the catalog and the runtime consumption sites.
namespace Sy.MonsterInn
{
    public static class InnAudioStems
    {
        public const string BgmInnNight = "g20_bgm_inn_night";             // AU-01 looping night BGM (lantern-sway hook
        public const string KnockSoft = "g20_knock_soft";                   // AU-02 buildup double-event knock
        public const string RevealStingCute = "g20_reveal_sting_cute";      // AU-03 reveal-door signature sting
        public const string ServeDish = "g20_serve_dish";                  // AU-04 serve same-tick triple feedback #1
        public const string CoinFortunePop = "g20_coin_fortune_pop";       // AU-05 checkout coin pop feedback #2
        public const string CodexUnlockChime = "g20_codex_unlock_chime";   // AU-06 codex unlock chime feedback #3
        public const string UpgradeLanternSwell = "g20_upgrade_lantern_swell"; // AU-07 inn upgrade beat
        public const string MorningSettleJingle = "g20_morning_settle_jingle"; // AU-08 morning stay settle overlay
        public const string WalkoutSoftPuff = "g20_walkout_soft_puff";      // AU-09 zero-punishment walkout puff
        public const string UiTapGeneric = "g20_ui_tap_generic";            // AU-10 generic UI / op1 seat tap
        public const string EventStingComedy = "g20_event_sting_comedy";    // AU-11 comedy event popup sting
    }
}
// InnBot.cs
// InnBot.cs - T011 bare-play bot: the E4 GrayboxBot ported onto the real engine
// flows. Twenty nights of the full loop per the frozen E4 scenario: arrivals -&gt;
// buildup/reveal -&gt; seat (op1) -&gt; serve (op2) -&gt; auto checkout + codex unlock,
// one sparse deterministic slip (night % 7 == 3 forgets the first guest's dish
// -&gt; honest zero-punishment walkout), nightly comedy event, a purely optional
// boost offer (never accepted), morning overnight settlement, one curve-costed
// cozy room after the settle (T012-c room rhythm: at most one per night), and a
// share card once the codex lineup is worth showing. Night zero closes the first-night
// service loop at the frozen T+60s promise and composes the share code on that
// beat (the T011 bot face the Redline11Check consumes). Pure C#, zero RNG, zero
// ads taken - the loop closes anyway.
using System;
using System.Collections.Generic;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;
    /// Deterministic bare-play bot on the real flows (InnFlow / ServeShift /
    /// CodexFlow / StayFlow / EventFlow / AdFlow / ShareCodec). Drives the same
    /// nightly loop the E4 graybox bot ran (120/117/3 caliber) but through the
    /// shipped runtime so the T011 gate asserts the red-line promises on the
    /// real engine, not a sim twin.
    /// &lt;/summary&gt;
    public sealed class InnBot
    {
        public readonly InnFlow Inn = new InnFlow();
        public readonly ServeShift Shift = new ServeShift();
        public readonly CodexFlow Codex = new CodexFlow();
        public readonly StayFlow Stay = new StayFlow();
        public readonly EventFlow Events = new EventFlow();
        public readonly AdFlow Ads = new AdFlow();
        public readonly ShareFlow Share = new ShareFlow();
```

```
public double Time;                    // master logical clock (all flows in lockstep)
public int Nights;
public int Arrivals;
public int ServedCount;
public double ServiceClosedAt = -1.0;   // night-0 clock: all guests + comedy + settle done
public double ClosureAt = -1.0;         // night-0 clock: the T+60s promise mark
public string ClosureCode;               // share code composed on the closure beat
public readonly List&lt;string&gt; Cards = new List&lt;string&gt;();
// T013-c FQ producer faces (additive, zero behavior change): per-night upgrade
// evidence for the next-goal pull census and the honest walkout-night count.
public readonly List&lt;double&gt; UpgradeWalletBefore = new List&lt;double&gt;();
public readonly List&lt;double&gt; UpgradeNextCost = new List&lt;double&gt;();
public int UpgradeRefusals;
public int WalkoutNights;
void Pump(double dt)
{
    Time += dt;
    Inn.Tick(dt);
    Shift.Tick(dt);
    Codex.Tick(dt);
    Stay.Tick(dt);
    Events.Tick(dt);
    Share.Tick(dt);
}
// Pump the master clock to an exact target in 0.05s slices (last slice trimmed).
void PumpTo(double target)
{
    while (Time + 1e-9 &lt; target)
    {
        double dt = target - Time;
        if (dt &gt; 0.05) dt = 0.05;
        Pump(dt);
    }
}
/// &lt;summary&gt;One deterministic night: six guests, comedy, offer, stay settle, card face.&lt;/summary&gt;
public void Night(int n)
{
    Nights++;
    double nightStart = Time;
    Inn.OpenInn(n);
    bool[] served = new bool[InnFrozen.GuestsPerNight];
    for (int i = 0; i &lt; InnFrozen.GuestsPerNight; i++)
    {
        int idx = n * InnFrozen.GuestsPerNight + i;
        Pump(0.50);                                  // arrival beat gap (graybox parity)
        Inn.BeginArrival(idx);                       // caption 0.15 -&gt; flicker 1.00 -&gt; reveal 2.50
        Pump(InnFrozen.RevealT);
        GuestSpec g = Inn.LastRevealed;
        Arrivals++;
        Shift.BeginGuest(g);
```

```
                Shift.Seat();                                   // op1 rides the reveal tick
                bool slipNight = (n % 7 == 3) &amp;&amp; (i == 0);      // sparse deterministic slip
                if (slipNight)
                {
                    Pump(InnFrozen.WalkPatience);               // auto walkout: honest loss, zero punishment
                    WalkoutNights++;
                }
                else
                {
                    Pump(InnFrozen.FirstServeT - InnFrozen.RevealT); // serve beat
                    Shift.Serve();                              // op2 same-tick triple
                    Pump(InnFrozen.MealDur);                    // auto checkout: coin pop + pay
                    Codex.UnlockEntry(g.Species);               // checkout-driven first-seen unlock
                    served[i] = true;
                    ServedCount++;
                }
            }
            Events.NightComedy(n);                              // three comedy families rotate, zero RNG
            Ads.OfferAd(AdFrozen.Categories[n % AdFrozen.Categories.Length], (n + 1) * 21600.0);
            int stayers = 0;
            for (int i = 0; i &lt; InnFrozen.GuestsPerNight; i++)
                if (i % 3 == 0 &amp;&amp; served[i]) stayers++;         // only served guests stay over
            Stay.MorningSettlement(stayers, n, Codex.Inn.Level, Shift.Wallet);   // idle/offline earnings face
            double upgradeWalletBefore = Shift.Wallet.Coins;                      // T013-c: next-goal census face
            double upgradeNextCost = InnFrozen.UpgradeCostFor(Codex.Inn.Level);   // clamps at the last rung once maxed
            bool innUpgraded = Codex.UpgradeInn(Shift.Wallet);   // S2 room rhythm (T012-c): at most one new cozy room p
            UpgradeWalletBefore.Add(upgradeWalletBefore);
            UpgradeNextCost.Add(upgradeNextCost);
            if (!innUpgraded) UpgradeRefusals++;
            string card = Share.TryShareCard(n, Codex);
            if (card != null) Cards.Add(card);
            if (n == 0)
            {
                // First-night closure beat: service done, loop closes at the frozen
                // T+60s promise mark, share code composes on that beat.
                ServiceClosedAt = Time - nightStart;
                PumpTo(nightStart + InnFrozen.NightClosureT);
                ClosureAt = Time - nightStart;
                ClosureCode = Share.ComposeCard(Codex);
            }
        }
        public void Run(int nights) { for (int n = 0; n &lt; nights; n++) Night(n); }
    }
}
// InnFlow.cs
// InnFlow.cs - S1 guest-flow core (T003 slice a): ports the frozen E4 graybox M1/M2/M3
// face into the real runtime. Open the inn, buildup (lamp flicker + soft knock, a double
// event on the same tick, all-ages capped), then the soft-cute reveal of the guest.
// Pure C# (no UnityEngine) so Editor gates can drive it headless; the view layer binds later.
// Every number is a frozen E4 anchor (GrayboxSim.cs) - zero re-tuning, zero RNG by design.
```

```
using System;
using System.Collections.Generic;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;Frozen E4 anchors: stage-1 hook timeline plus economy baselines.&lt;/summary&gt;
    public static class InnFrozen
    {
        public const double CaptionT = 0.15;      // hook caption (red line 11, T+0-3s)
        public const double FlickerT = 1.00;      // lamps flicker + soft knock (buildup, full-age cap)
        public const double RevealT = 2.50;       // door opens on the soft fluffy reveal (first wow)
        public const double FirstServeT = 4.00;   // first serve tap (first core op, T004)
        public const double CodexUnlockT = 6.00;  // first codex entry (T005)
        public const double UpgradeT = 10.00;     // first inn upgrade (T005)
        public const double NightClosureT = 60.00; // first-night loop + share-code beat (T+60s promise)
        public const double MealDur = 2.00;       // serve -&gt; meal done (T004)
        public const double WalkPatience = 6.00;  // un-served guest gives up and leaves (T004)
        public const double UpgradeCost = 4.0;
        public const double PayPerGuest = 2.0;
        public const double StayRatePerHour = 1.0;
        public const double StayCapPerNight = 20.0;
        public const double AwayHours = 8.0;
        public const int GuestsPerNight = 6;
        public const int RosterSize = 12;
        // S2 rarity tiering (T012-b): three frozen tiers 6/4/2 across the 12 roster,
        // pay layered 2.0/3.0/5.0 - the E4 re-freeze face (17/17) mirrors numeric pack
        // section 1.1. The first two guests are common, so every Stage1 beat keeps its
        // flat-pay anchor exactly (first serve, first upgrade at 10.00).
        public static readonly string[] RarityTiers = { "common", "rare", "epic" };
        public static readonly int[] RosterRarity = { 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 2, 2 };
        public static readonly double[] PayByRarity = { PayPerGuest, 3.0, 5.0 };
        public static double PayForRosterIndex(int rosterIndex)
        {
            int slot = ((rosterIndex % RosterSize) + RosterSize) % RosterSize;
            return PayByRarity[RosterRarity[slot]];
        }
        public static string RarityOfSpecies(string species)
        {
            for (int i = 0; i &lt; RosterSize; i++)
                if (CodexRoster[i] == species) return RarityTiers[RosterRarity[i]];
            return null;
        }
        // S2 room-unlock rhythm (T012-c): the multi-level upgrade curve 4/8/14/22/32/44
        // (deltas +4/+6/+8/+10/+12, sum 124) with the full build-out at level 7 - a maxed
        // inn refuses further upgrades silently (zero mutation, zero events). The stay
        // ceiling grows with the level: cap(L) = 20 + 4*(L-1) (numeric pack 1.2/1.4).
        public const int InnMaxLevel = 7;
        public static readonly double[] UpgradeCostByLevel = { 4.0, 8.0, 14.0, 22.0, 32.0, 44.0 };
        public const double StayCapSlope = 4.0;
        public static double UpgradeCostFor(int level)
        {
```

```
            int i = level - 1;   // index 0 = the L1-&gt;L2 step (keeps the S1 anchor 4.0)
            if (i &lt; 0) i = 0;
            if (i &gt;= UpgradeCostByLevel.Length) i = UpgradeCostByLevel.Length - 1;
            return UpgradeCostByLevel[i];
        }
        public static double StayCapFor(int level)
        {
            return StayCapPerNight + StayCapSlope * (level - 1);
        }
        // E3#3 all-ages vocabulary: cozy-horror tokens stay cute and folk-flavored,
        // superstition is comedy-abstracted, zero real-religion / horror-grade tokens.
        public static readonly string[] CuteVocabulary =
        {
            "lantern_warm_glow", "night_indigo_sky", "cute_fluffy_yokai", "soft_knock_knock",
            "paper_lantern_bob", "sweet_dumpling_steam", "porcelain_teacup", "wooden_inn_sign",
            "lucky_coin_pouch", "blind_box_fortune_slip"
        };
        public static readonly string[] ForbiddenTokens =
        {
            "blood", "gore", "jumpscare", "horror_face", "real_religion",
            "burning_paper_offering", "funeral", "cruel_scream", "dark_ritual", "possession"
        };
        // E3#5 zero-punishment face: loss tokens that must never appear in any event
        // stream (a walkout is honest loss, never a penalty). Frozen from E4 GrayboxSim.
        public static readonly string[] PunishTokens =
        {
            "punish", "penalty", "codex_reset", "fine_", "score_down"
        };
        // 12 yokai species, zhiguai folk-lore flavor, all cute, zero real IP names.
        public static readonly string[] CodexRoster =
        {
            "radish_spirit", "lantern_imp", "teapot_elder", "kite_kid",
            "inkstone_guest", "dumpling_fairy", "scissors_hopper", "bell_wisp",
            "mat_roller", "straw_cloak_wanderer", "candle_twins", "cricket_singer"
        };
    }
    public sealed class InnEvent
    {
        public readonly double T;
        public readonly string Type;
        public readonly string Detail;
        public InnEvent(double t, string type, string detail) { T = t; Type = type; Detail = detail; }
    }
    public sealed class GuestSpec
    {
        public string Species;       // codex roster token
        public string Rarity;        // S2 tier tag (common / rare / epic, T012-b)
        public bool StayOvernight;    // picks the night-stay room (offline earnings face)
        public double Pay;
    }
```

```
/// &lt;summary&gt;
/// M1 open -&gt; M2 buildup -&gt; M3 reveal machine for one guest arrival. The caller pumps
/// Tick(dt) on a logical clock; caption, the buildup double event and the reveal fire
/// themselves at the frozen anchors (the product owns its timeline, not the caller).
/// &lt;/summary&gt;
public sealed class InnFlow
{
    public double Time;   // arrival-local logical seconds
    public readonly List&lt;InnEvent&gt; Events = new List&lt;InnEvent&gt;();
    public int Arrivals;
    private bool _captionFired;
    private bool _buildupFired;
    private bool _revealFired;
    private int _arrivalIndex = -1;
    public bool RevealDone { get { return _revealFired; } }
    public GuestSpec LastRevealed { get; private set; }
    public void Log(string type, string detail) { Events.Add(new InnEvent(Time, type, detail)); }
    public double TimeOf(string type)
    {
        for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) return Events[i].T;
        return -1.0;
    }
    public int CountEvents(string type)
    {
        int n = 0;
        for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) n++;
        return n;
    }
    // M1: the wooden sign flips over - the inn is open for yokai guests tonight.
    public void OpenInn(int nightIndex)
    {
        Log("inn_open", "night=" + nightIndex.ToString(System.Globalization.CultureInfo.InvariantCulture)
            + " the wooden sign flips over: yokai guests welcome");
    }
    // Starts one guest arrival; pump Tick(dt) until RevealDone, then read LastRevealed.
    public void BeginArrival(int arrivalIndex)
    {
        _arrivalIndex = arrivalIndex;
        Time = 0.0;
        _captionFired = false;
        _buildupFired = false;
        _revealFired = false;
    }
    // Advance the arrival clock; fires caption / buildup / reveal at the frozen anchors.
    public void Tick(double dt)
    {
        if (_revealFired || _arrivalIndex &lt; 0) return;
        Time += dt;
        if (!_captionFired &amp;&amp; Time + 1e-9 &gt;= InnFrozen.CaptionT)
        {
```

```
                _captionFired = true;
                Log("caption", "ui_slogan");
            }
            if (!_buildupFired &amp;&amp; Time + 1e-9 &gt;= InnFrozen.FlickerT)
            {
                _buildupFired = true;
                // M2 buildup double event: lamp flicker + soft knock on the same tick.
                Log("lamps_flicker", "lanterns sway and dim, a soft knock-knock at the door");
                Log("sfx_knock", "gentle wood tap");
                InnAudioHooks.OnKnock();  // T010: same-tick buildup knock hook
            }
            if (!_revealFired &amp;&amp; Time + 1e-9 &gt;= InnFrozen.RevealT)
            {
                _revealFired = true;
                // M3 reveal: deterministic roster rotation, zero RNG.
                string species = InnFrozen.CodexRoster[_arrivalIndex % InnFrozen.RosterSize];
                Log("door_opens", "a " + species + " bows politely and waddles in");
                InnAudioHooks.OnReveal();  // T010: same-tick reveal sting hook (first wow)
                LastRevealed = new GuestSpec
                {
                    Species = species,
                    Rarity = InnFrozen.RarityOfSpecies(species),
                    StayOvernight = _arrivalIndex % 3 == 0,
                    Pay = InnFrozen.PayForRosterIndex(_arrivalIndex)
                };
                Arrivals++;
            }
        }
    }
}
// InnGuide.cs
// InnGuide.cs - T009-c per-second tutorial table + first-night guide walker.
// GuideFrozen is the frozen seven-beat table (requirements doc section 1 beat
// table, E4 Stage1 frozen): the direct input for the T011 Redline11Check
// per-second mapping (E4 assertions 1-5) and the bind face for the T010 view
// hints. InnGuide walks the frozen E4 Stage1 opening night on the REAL flows
// (InnFlow / ServeShift / CodexFlow) with zero new beats: hook window (caption
// 0.15 -&gt; buildup 1.00 double event -&gt; reveal 2.50 first wow), op1 rides the
// reveal tick, op2 serve triple at 4.00, checkout at 6.00 (+2 coin pop + first
// codex unlock), a second guest pipelined from the first serve beat so its
// meal settles at 10.00 and the wallet hits the frozen 4.0 upgrade cost, and
// the first upgrade lands on that same beat. Pure C# (no UnityEngine) so the
// Editor gate drives it headless; the view layer binds in T010/T011. Zero RNG.
using System;
using System.Collections.Generic;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;One frozen per-second beat row (night clock, logical seconds).&lt;/summary&gt;
    public struct GuideBeat
    {
```

```
    public readonly double T;
    public readonly string Key;
    public readonly string Detail;
    public GuideBeat(double t, string key, string detail) { T = t; Key = key; Detail = detail; }
}
/// &lt;summary&gt;
/// Frozen seven-beat per-second table (E4 Stage1, zero re-tuning). Every time is
/// sourced from InnFrozen so the table and the flow anchors can never drift apart.
/// &lt;/summary&gt;
public static class GuideFrozen
{
    public const double HookWindowEnd = 3.00;         // T+0-3s buildup hook window
    public const double FirstOpsWindowStart = 3.00;   // T+3-10s first-ops window
    public const double FirstOpsWindowEnd = 10.00;
    public const double SignalsDeadline = 10.00;      // three addictive signals inside 10s
    public const int BeatCount = 7;
    public static readonly GuideBeat[] Beats =
    {
        new GuideBeat(InnFrozen.CaptionT, "caption",
            "T+0.15 the hook caption lands first (red line 11)"),
        new GuideBeat(InnFrozen.FlickerT, "buildup",
            "T+1.00 lamps flicker + soft knock on the same tick (full-age cap)"),
        new GuideBeat(InnFrozen.RevealT, "reveal",
            "T+2.50 the door opens on the soft fluffy reveal (first wow)"),
        new GuideBeat(InnFrozen.FirstServeT, "serve",
            "T+4.00 op2 serve: same-tick triple (tap + sfx + served pop)"),
        new GuideBeat(InnFrozen.CodexUnlockT, "codex_unlock",
            "T+6.00 checkout: +2 coin pop + first codex unlock popup"),
        new GuideBeat(InnFrozen.UpgradeT, "inn_upgrade",
            "T+10.00 first inn upgrade (growth spine first beat)"),
        new GuideBeat(InnFrozen.NightClosureT, "night_closure",
            "T+60s first night service loop closes + share code (T011 bot face)"),
    };
}
/// &lt;summary&gt;
/// First-night guide walker: pumps one master night clock across the real flows
/// in lockstep and records each beat's landing time on the night clock (BeatAt,
/// -1 = not walked) plus the three addictive-signal times. Deterministic by
/// construction - the walkthrough is the tutorial reference timeline the T010
/// view binds for live hints and the gate asserts against the frozen promises.
/// &lt;/summary&gt;
public sealed class InnGuide
{
    public double Time;   // master night clock (logical seconds)
    public readonly List&lt;InnEvent&gt; Events = new List&lt;InnEvent&gt;();
    public readonly double[] BeatAt = new double[GuideFrozen.BeatCount];
    public double RevealAt, CoinAt, CodexAt;   // three addictive-signal times
    public bool Walked;
    InnFlow _inn;
    ServeShift _shift;
```

```
CodexFlow _codex;
public void Log(string type, string detail) { Events.Add(new InnEvent(Time, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) return Events[i].T;
    return -1.0;
}
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) n++;
    return n;
}
/// &lt;summary&gt;Three addictive signals all landed inside the 10s deadline.&lt;/summary&gt;
public bool SignalsWithinDeadline
{
    get
    {
        return RevealAt &gt; 0.0 &amp;&amp; CoinAt &gt; 0.0 &amp;&amp; CodexAt &gt; 0.0
            &amp;&amp; RevealAt &lt;= GuideFrozen.SignalsDeadline
            &amp;&amp; CoinAt &lt;= GuideFrozen.SignalsDeadline
            &amp;&amp; CodexAt &lt;= GuideFrozen.SignalsDeadline;
    }
}
void MarkBeat(int index, double t, string note)
{
    BeatAt[index] = t;
    Log("guide_beat", GuideFrozen.Beats[index].Key + " @ "
        + t.ToString("0.00", System.Globalization.CultureInfo.InvariantCulture) + " " + note);
}
void Step(double dt)
{
    Time += dt;
    _inn.Tick(dt);
    _shift.Tick(dt);
    _codex.Tick(dt);
}
// Pump the master clock to an exact target in 0.05s slices (last slice trimmed).
void PumpTo(double target)
{
    while (Time + 1e-9 &lt; target)
    {
        double dt = target - Time;
        if (dt &gt; 0.05) dt = 0.05;
        Step(dt);
    }
}
/// &lt;summary&gt;
/// Deterministic E4 Stage1 opening-night walkthrough on the given live flows
/// (fresh instances for the gate; the InnSaveHost set at runtime). Zero RNG.
```

```
    /// &lt;/summary&gt;
    public void Walk(InnFlow inn, ServeShift shift, CodexFlow codex)
    {
        _inn = inn;
        _shift = shift;
        _codex = codex;
        Time = 0.0;
        Walked = false;
        RevealAt = CoinAt = CodexAt = 0.0;
        for (int i = 0; i &lt; BeatAt.Length; i++) BeatAt[i] = -1.0;
        Log("guide_open", "first-night tutorial: seven frozen beats, three signals inside ten seconds");
        // ---- beats 1-3, hook window: the arrival machine self-fires the anchors ----
        inn.BeginArrival(0);
        PumpTo(InnFrozen.RevealT);                    // caption 0.15, buildup 1.00, reveal 2.50
        MarkBeat(0, inn.TimeOf("caption"), "hook caption");
        MarkBeat(1, inn.TimeOf("lamps_flicker"), "buildup double event");
        MarkBeat(2, inn.TimeOf("door_opens"), "reveal first wow");
        RevealAt = BeatAt[2];
        // op1 rides the reveal tick (E4 Stage1 face), op2 lands at the serve beat.
        GuestSpec g0 = inn.LastRevealed;
        shift.BeginGuest(g0);
        shift.Seat();
        PumpTo(InnFrozen.FirstServeT);
        shift.Serve();                                // same-tick triple at 4.00
        MarkBeat(3, Time, "first serve triple");
        // ---- second guest pipelined from the serve beat: its arrival clock starts
        // at 4.00, so its reveal lands at 6.50, its serve at 8.00 and its meal
        // settles at 10.00 - exactly when the wallet reaches the upgrade cost ----
        double guestTwo = InnFrozen.FirstServeT;
        PumpTo(guestTwo);
        inn.BeginArrival(1);
        PumpTo(InnFrozen.CodexUnlockT);               // 6.00: guest-1 checkout
        MarkBeat(4, Time, "checkout: coin pop + first codex unlock");
        CoinAt = _shift.TimeOf("coin_pop");
        _codex.UnlockEntry(g0.Species);              // checkout-driven, same beat
        CodexAt = _codex.TimeOf("codex_unlock");
        PumpTo(guestTwo + InnFrozen.RevealT);        // 6.50: guest-2 reveal
        GuestSpec g1 = inn.LastRevealed;
        _shift.BeginGuest(g1);
        _shift.Seat();                                // op1 rides the reveal tick
        PumpTo(guestTwo + InnFrozen.FirstServeT);    // 8.00: guest-2 serve
        _shift.Serve();
        PumpTo(guestTwo + InnFrozen.CodexUnlockT);   // 10.00: settle, wallet hits 4.0
        _codex.UnlockEntry(g1.Species);              // second codex entry, 10.00
        _codex.UpgradeInn(_shift.Wallet);            // first upgrade on the same beat
        MarkBeat(5, Time, "first upgrade");
        // beat 7 (night_closure, T+60s) is the T011 bot face - stays unwalked here.
        Walked = true;
        Log("guide_done", "tutorial beats closed: signals at "
            + RevealAt.ToString("0.00", System.Globalization.CultureInfo.InvariantCulture) + "/"
```

```
                + CoinAt.ToString("0.00", System.Globalization.CultureInfo.InvariantCulture) + "/"
                + CodexAt.ToString("0.00", System.Globalization.CultureInfo.InvariantCulture));
        }
    }
}
// InnPageHost.cs
// InnPageHost.cs - scene-side page mount: builds its page skeleton on Start via
// the shared InnPages builders (single source - the gate reads the same tables).
// The boot page (S99_Boot) also binds the save call site through the service
// locator; services may register after this host starts, so the bind retries a
// few frames instead of silently skipping (null-safe, never throws).
using System.Collections;
using UnityEngine;
namespace Sy.MonsterInn
{
    public sealed class InnPageHost : MonoBehaviour
    {
        public string pageId = PageIds.Boot;
        bool _built;
        void Start()
        {
            if (_built) return;
            _built = true;
            InnPages.BuildPage(pageId, transform);
            if (pageId == PageIds.Boot) StartCoroutine(BindWhenReady());
            if (pageId == PageIds.InnHall) EnsureAudioHost();
        }
        // T010: the hall page owns the audio face (BGM bed + same-tick hook playback).
        // Idempotent and null-safe - a missing catalog stays silent, never throws.
        void EnsureAudioHost()
        {
            if (FindObjectOfType&lt;InnAudioHost&gt;() != null) return;
            var go = new GameObject("InnAudioHost");
            go.transform.SetParent(transform, false);
            go.AddComponent&lt;InnAudioHost&gt;();
        }
        IEnumerator BindWhenReady()
        {
            int guard = 0;
            while (!InnSaveHost.BindBootFromServices() &amp;&amp; guard &lt; 30)
            {
                guard++;
                yield return null;
            }
        }
    }
}
// InnPages.cs
// InnPages.cs - T009-b page skeleton: frozen five-page registry + programmatic UGUI.
// X027 iron rule: art-first (ArtRuntime stem) with a procedural color fallback, so
```

```
// the page renders either way and no headless gate ever depends on art presence.
// Spec source = G20 requirements doc section 3 (frozen): pages S99_Boot /
// S01_InnHall / S01a_MorningSettle (overlay mounted INSIDE S01) / S02_Codex /
// S03_Share; per-page key elements are the frozen tables below - the gate reads
// the same tables, so inventory is single-sourced (no drift between spec, build
// and gate). U184 day-one: every page canvas carries CanvasScaler
// (ScaleWithScreenSize 750x1334 portrait) + GraphicRaycaster.
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;Frozen page registry (requirements doc section 3 page list).&lt;/summary&gt;
    public static class PageIds
    {
        public const string Boot = "S99_Boot";
        public const string InnHall = "S01_InnHall";
        public const string MorningSettle = "S01a_MorningSettle"; // overlay, mounted inside S01
        public const string Codex = "S02_Codex";
        public const string Share = "S03_Share";
        public static readonly string[] All =
        {
            Boot, InnHall, MorningSettle, Codex, Share
        };
        /// &lt;summary&gt;Pages that own a scene file (the S01a overlay rides inside S01).&lt;/summary&gt;
        public static readonly string[] SceneBacked =
        {
            Boot, InnHall, Codex, Share
        };
    }
    /// &lt;summary&gt;Frozen per-page key elements (the gate reads these - zero-gap source).&lt;/summary&gt;
    public static class PageNodes
    {
        public const string DoorFlickerSlot = "door_flicker_slot";
        public const string SeatGrid = "seat_grid";
        public const string FortuneHud = "fortune_hud";
        public const string CodexEntry = "codex_entry";
        public const string ShareEntry = "share_entry";
        public const string AdButtons = "ad_buttons";
        public const string StayBubble = "stay_bubble";
        public const string AdBtnServiceDouble = "ad_btn_service_double";
        public const string AdBtnRareSummon = "ad_btn_rare_summon";
        public const string AdBtnOfflineClaim = "ad_btn_offline_claim";
        public const string OfflineDetail = "offline_detail";
        public const string ClaimButton = "claim_button";
        public const string SettleCaption = "settle_caption";
        public const string CodexWall = "codex_wall";
        public const string RarityBadges = "rarity_badges";
        public const string ZeroPenaltyCopy = "zero_penalty_copy";
```

```
    public const string CodexCellPrefix = "codex_cell_";
    public const string ShareCodeText = "share_code_text";
    public const string CopyButton = "copy_button";
    public const string DecodeInput = "decode_input";
    public const string LineupPreview = "lineup_preview";
    /// &lt;summary&gt;Page -&gt; required key elements (requirements doc section 3, frozen).&lt;/summary&gt;
    public static readonly Dictionary&lt;string, string[]&gt; Spec = new Dictionary&lt;string, string[]&gt;
    {
        {
            PageIds.InnHall,
            new[] { DoorFlickerSlot, SeatGrid, FortuneHud, CodexEntry, ShareEntry, AdButtons, StayBubble }
        },
        {
            PageIds.MorningSettle,
            new[] { OfflineDetail, ClaimButton, SettleCaption }
        },
        {
            PageIds.Codex,
            new[] { CodexWall, RarityBadges, ZeroPenaltyCopy }
        },
        {
            PageIds.Share,
            new[] { ShareCodeText, CopyButton, DecodeInput, LineupPreview }
        },
    };
}
/// &lt;summary&gt;Programmatic page builder (X027: gates and runtime hosts share one source).&lt;/summary&gt;
public static class InnPages
{
    // brand faces: night-indigo hall + lantern-amber accents + paper cream
    public static readonly Color NightIndigo = new Color(0.09f, 0.07f, 0.14f, 1f);
    public static readonly Color LanternAmber = new Color(1f, 0.62f, 0.28f, 1f);
    public static readonly Color PaperCream = new Color(0.96f, 0.92f, 0.82f, 1f);
    static readonly Dictionary&lt;string, Sprite&gt; _faces = new Dictionary&lt;string, Sprite&gt;();
    /// &lt;summary&gt;Art-first face (X027): the shipped art stem wins, else a procedural
    /// color block - the page renders either way.&lt;/summary&gt;
    public static Sprite Face(string stem, Color color, int px)
    {
        Sprite art = ArtRuntime.Get(stem);
        if (art != null) return art;
        string key = stem + "#" + px;
        Sprite cached;
        if (_faces.TryGetValue(key, out cached) &amp;&amp; cached != null) return cached;
        var tex = new Texture2D(px, px, TextureFormat.RGBA32, false);
        var block = new Color[px * px];
        for (int i = 0; i &lt; block.Length; i++) block[i] = color;
        tex.SetPixels(block);
        tex.Apply(false, false);
        tex.name = "sy_procedural_" + px;
        var made = Sprite.Create(tex, new Rect(0, 0, px, px), new Vector2(0.5f, 0.5f), px);
```

```
        made.name = tex.name;
        _faces[key] = made;
        return made;
    }
    /// &lt;summary&gt;Builds every page (S01a rides inside S01). Gate + host entry.&lt;/summary&gt;
    public static Transform BuildAll(Transform parent)
    {
        var root = new GameObject("inn_pages").transform;
        if (parent != null) root.SetParent(parent, false);
        BuildPage(PageIds.Boot, root);
        BuildPage(PageIds.InnHall, root);
        BuildPage(PageIds.Codex, root);
        BuildPage(PageIds.Share, root);
        return root;
    }
    /// &lt;summary&gt;Builds one page skeleton under parent; unknown ids return null.&lt;/summary&gt;
    public static Transform BuildPage(string pageId, Transform parent)
    {
        switch (pageId)
        {
            case PageIds.Boot: return BuildBoot(parent);
            case PageIds.InnHall: return BuildInnHall(parent);
            case PageIds.Codex: return BuildCodex(parent);
            case PageIds.Share: return BuildShare(parent);
            default: return null;
        }
    }
    /// &lt;summary&gt;Depth-first child lookup by name (gates read the built tree).&lt;/summary&gt;
    public static Transform Find(Transform root, string name)
    {
        if (root == null) return null;
        if (root.name == name) return root;
        for (int i = 0; i &lt; root.childCount; i++)
        {
            var hit = Find(root.GetChild(i), name);
            if (hit != null) return hit;
        }
        return null;
    }
    // ---- page roots ----
    static Transform PageRoot(string pageId, Transform parent, bool overlay)
    {
        var go = new GameObject(pageId);
        if (parent != null) go.transform.SetParent(parent, false);
        var canvas = go.AddComponent&lt;Canvas&gt;();
        canvas.renderMode = RenderMode.ScreenSpaceOverlay;
        var scaler = go.AddComponent&lt;CanvasScaler&gt;();
        scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize; // U184 day-one
        scaler.referenceResolution = new Vector2(750f, 1334f);           // portrait lock (S0)
        scaler.matchWidthOrHeight = 0.5f;
```

```
    go.AddComponent&lt;GraphicRaycaster&gt;();                                // U184 day-one input chain
    if (overlay)
    {
        // A nested overlay canvas already rides INSIDE its host page's
        // SafeArea container, so the host fitter squeezes its faces. A
        // second fitter here would re-apply Screen.safeArea against a
        // non-driven nested rect and misplace the overlay the moment
        // it activates (doc-17 section 4: one SafeArea per canvas).
        go.SetActive(false); // the morning overlay waits for its settle moment
        return go.transform;
    }
    // BR-G20-1 safe-area mount (B864 L2 sweep RED -&gt; B641 G08 pattern):
    // every page face parents under one full-stretch SafeArea child, so
    // Screen.safeArea (notch, corner radius) squeezes the play field
    // instead of cropping must-see faces. Where safeArea equals the
    // full screen (editor gates, batchmode) offsets stay zero, so the
    // frozen anchors and every pixel probe keep their positions.
    var safeGo = new GameObject("SafeArea", typeof(RectTransform));
    safeGo.transform.SetParent(go.transform, false);
    var safeRt = (RectTransform)safeGo.transform;
    safeRt.anchorMin = Vector2.zero;
    safeRt.anchorMax = Vector2.one;
    safeRt.offsetMin = safeRt.offsetMax = Vector2.zero;
    safeGo.AddComponent&lt;SafeAreaFitter&gt;();
    return safeGo.transform;
}
static Transform BuildBoot(Transform parent)
{
    var root = PageRoot(PageIds.Boot, parent, false);
    var logo = Node(root, "logo_slot", "ui_ico_set", PaperCream);
    Anchor(logo, 0.5f, 0.5f, 0f, 60f, 160f, 160f);
    var caption = TextNode(root, "loading_caption", "boot_loading", 30);
    Anchor(caption, 0.5f, 0.5f, 0f, -60f, 500f, 60f);
    var progress = Node(root, "progress_face", "ui_ico_set", LanternAmber);
    Anchor(progress, 0.5f, 0.5f, 0f, -140f, 320f, 24f);
    return root;
}
static Transform BuildInnHall(Transform parent)
{
    var root = PageRoot(PageIds.InnHall, parent, false);
    var bg = Node(root, "hall_bg", "inn_hall_bg", NightIndigo);
    Anchor(bg, 0.5f, 0.5f, 0f, 0f, 730f, 1334f);
    var door = Node(root, PageNodes.DoorFlickerSlot, "lantern_lamp", LanternAmber);
    Anchor(door, 0.5f, 0f, 0f, 96f, 128f, 160f);
    var seats = Node(root, PageNodes.SeatGrid, "inn_hall_bg", NightIndigo);
    Anchor(seats, 0.5f, 0.5f, 0f, -80f, 640f, 480f);
    var hud = Node(root, PageNodes.FortuneHud, "ui_ico_set", LanternAmber);
    Anchor(hud, 0f, 1f, 154f, -56f, 260f, 64f); // pivot law: center the 260x64 face 24px inside the top-left co
    var codexEntry = Node(root, PageNodes.CodexEntry, "ui_ico_set", PaperCream);
    Anchor(codexEntry, 1f, 1f, -56f, -56f, 64f, 64f); // 64x64 centered 24px inside the top-right corner
```

```
    var shareEntry = Node(root, PageNodes.ShareEntry, "ui_ico_set", PaperCream);
    Anchor(shareEntry, 1f, 1f, -56f, -140f, 64f, 64f); // same inset, 84px below the codex entry
    var ads = Node(root, PageNodes.AdButtons, "ui_ico_set", NightIndigo);
    Anchor(ads, 0.5f, 0f, 0f, 60f, 480f, 72f); // 480x72 centered 24px above the bottom edge
    Node(ads, PageNodes.AdBtnServiceDouble, "ui_ico_set", LanternAmber);
    Node(ads, PageNodes.AdBtnRareSummon, "ui_ico_set", LanternAmber);
    Node(ads, PageNodes.AdBtnOfflineClaim, "ui_ico_set", LanternAmber);
    var bubble = Node(root, PageNodes.StayBubble, "guest_units_roster12", PaperCream);
    Anchor(bubble, 0f, 0.5f, 124f, 120f, 200f, 72f); // 200x72 centered 24px inside the left edge
    // S01a morning overlay rides INSIDE S01 (requirements doc: mounted on S01)
    var overlay = PageRoot(PageIds.MorningSettle, root, true);
    var panel = Node(overlay, "settle_panel", "morning_settle_panel", PaperCream);
    Anchor(panel, 0.5f, 0.5f, 0f, 0f, 560f, 420f);
    var detail = Node(overlay, PageNodes.OfflineDetail, "morning_settle_panel", NightIndigo);
    Anchor(detail, 0.5f, 0.5f, 0f, 40f, 480f, 160f);
    var claim = Node(overlay, PageNodes.ClaimButton, "ui_ico_set", LanternAmber);
    Anchor(claim, 0.5f, 0.5f, 0f, -120f, 320f, 72f);
    var caption = TextNode(overlay, PageNodes.SettleCaption, "morning_caption", 28);
    Anchor(caption, 0.5f, 0.5f, 0f, 150f, 520f, 60f);
    return root;
}
static Transform BuildCodex(Transform parent)
{
    var root = PageRoot(PageIds.Codex, parent, false);
    var wall = Node(root, PageNodes.CodexWall, "codex_wall", NightIndigo);
    Anchor(wall, 0.5f, 0.5f, 0f, 0f, 680f, 900f);
    for (int i = 0; i &lt; 12; i++)
        Node(wall, PageNodes.CodexCellPrefix + i.ToString("00"), "guest_units_roster12", PaperCream);
    var badges = Node(root, PageNodes.RarityBadges, "ui_ico_set", LanternAmber);
    Anchor(badges, 0.5f, 1f, 0f, -24f, 680f, 48f);
    var copy = TextNode(root, PageNodes.ZeroPenaltyCopy, "codex_zero_penalty", 26);
    Anchor(copy, 0.5f, 0f, 0f, 32f, 680f, 56f);
    return root;
}
static Transform BuildShare(Transform parent)
{
    var root = PageRoot(PageIds.Share, parent, false);
    var frame = Node(root, "share_frame", "share_card_frame", NightIndigo);
    Anchor(frame, 0.5f, 0.5f, 0f, 0f, 560f, 800f);
    var code = TextNode(root, PageNodes.ShareCodeText, "MI1-", 34);
    Anchor(code, 0.5f, 0.5f, 0f, 220f, 520f, 64f);
    var copyBtn = Node(root, PageNodes.CopyButton, "ui_ico_set", LanternAmber);
    Anchor(copyBtn, 0.5f, 0.5f, 0f, 140f, 320f, 64f);
    var input = Node(root, PageNodes.DecodeInput, "ui_ico_set", PaperCream);
    Anchor(input, 0.5f, 0.5f, 0f, -60f, 520f, 64f);
    var preview = Node(root, PageNodes.LineupPreview, "guest_units_roster12", PaperCream);
    Anchor(preview, 0.5f, 0.5f, 0f, -220f, 480f, 280f);
    return root;
}
// ---- primitives ----
```

```
    static Font _uiFont;
    /// &lt;summary&gt;U184 real-device CJK face (B757 font chain): LegacyRuntime.ttf
    /// ships no zh glyphs, so every zh display line would render blank on a
    /// device. The shipped NotoSansSC census subset (Resources) is the display
    /// face; any miss (asset absent) degrades to the built-in face - the
    /// null-safe law, never a crash. Gate side: AudioCheck D1 pins the wired
    /// face, D2 pins zero missing glyphs over the whole copy table.&lt;/summary&gt;
    static Font UiFont()
    {
        if (_uiFont != null) return _uiFont;
        var shipped = Resources.Load&lt;Font&gt;(LocalizeService.ShippedFontName);
        if (shipped != null) { _uiFont = shipped; return shipped; }
        _uiFont = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
        return _uiFont;
    }
    static Transform Node(Transform parent, string name, string stem, Color tint)
    {
        var go = new GameObject(name);
        go.transform.SetParent(parent, false);
        var rt = go.AddComponent&lt;RectTransform&gt;();
        rt.sizeDelta = new Vector2(96f, 96f);
        var img = go.AddComponent&lt;Image&gt;();
        img.sprite = Face(stem, tint, 48);
        return go.transform;
    }
    static Transform TextNode(Transform parent, string name, string keyText, int size)
    {
        var go = new GameObject(name);
        go.transform.SetParent(parent, false);
        var rt = go.AddComponent&lt;RectTransform&gt;();
        rt.sizeDelta = new Vector2(400f, 60f);
        var text = go.AddComponent&lt;Text&gt;();
        text.font = UiFont(); // shipped zh subset face, built-in only as the null-safe fallback
        text.fontSize = size;
        text.alignment = TextAnchor.MiddleCenter;
        text.color = PaperCream;
        // T010 i18n routing: keys resolve through LocalizeService - zero hardcoded copy
        text.text = LocalizeService.Get(keyText);
        return go.transform;
    }
    static void Anchor(Transform t, float ax, float ay, float ox, float oy, float w, float h)
    {
        var rt = t as RectTransform;
        if (rt == null) return;
        rt.anchorMin = new Vector2(ax, ay);
        rt.anchorMax = new Vector2(ax, ay);
        rt.anchoredPosition = new Vector2(ox, oy);
        rt.sizeDelta = new Vector2(w, h);
    }
}
```

```
}
// InnSaveHost.cs
// InnSaveHost.cs - T009-b save call sites: the runtime facade that owns the six
// flow instances and binds them to storage. Boot face restores (single restore
// path, null-safe first boot), close face persists (single capture path). The
// S99_Boot page host binds through the service locator, so the save wire is
// actually alive at runtime (facade consumption face - not a dead API).
using Sy.Core;
namespace Sy.MonsterInn
{
    public static class InnSaveHost
    {
        // canonical runtime flow set (the T011 game host consumes these)
        public static readonly InnFlow Inn = new InnFlow();
        public static readonly ServeShift Shift = new ServeShift();
        public static readonly CodexFlow Codex = new CodexFlow();
        public static readonly StayFlow Stay = new StayFlow();
        public static readonly EventFlow Events = new EventFlow();
        public static readonly AdFlow Ads = new AdFlow();
        /// &lt;summary&gt;True once the boot call site has run at least once.&lt;/summary&gt;
        public static bool Bound { get; private set; }
        /// &lt;summary&gt;Boot call site: storage -&gt; live flows (null payload = first
        /// boot, live values untouched - SaveWire.Restore is the single path).&lt;/summary&gt;
        public static void BindBootFrom(ISaveService save)
        {
            SaveWire.Restore(SaveWire.Load(save), Inn, Shift, Codex, Stay, Events, Ads);
            Bound = true;
        }
        /// &lt;summary&gt;Locator face for the boot page: binds through ServiceLocator
        /// (null-safe: no service registered yet = silent skip, X027 wiring contract).&lt;/summary&gt;
        public static bool BindBootFromServices()
        {
            if (!ServiceLocator.TryGet&lt;ISaveService&gt;(out var save)) return false;
            BindBootFrom(save);
            return true;
        }
        /// &lt;summary&gt;Close call site: live flows -&gt; storage (SaveWire.Save is the single path).&lt;/summary&gt;
        public static void PersistTo(ISaveService save)
        {
            SaveWire.Save(save, Inn, Shift, Codex, Stay, Events, Ads);
        }
    }
}
// LocalizeService.cs
// LocalizeService.cs - T010 frozen zh-Hans copy table (U114 L0: externalized keys,
// zero hardcoded copy). Single source of player-visible text: runtime code routes
// every display string through Get(key). BR-G20-2 (U262 L2 sweep, iron law 8:
// English-only source strings) moved the zh table out of compiled code into the
// data file Assets/_Game/Resources/Texts_zhCN.json (the G05 RodTexts pattern -
// data, not source), loaded lazily through Resources.Load so the WebGL face
```

```
// ships the table inside the build. ui_slogan and morning_caption are verbatim
// from the requirements doc (frozen copy); the key list extends the doc's
// section-6 i18n inventory ("boot_loading" and "codex_zero_penalty" are
// page-local additions under the doc's open "etc." set). T012-d event bank:
// three families x ten entries, one stylistic signature per family (G10
// style-signature law) - yokai news anchor / warm documentary / wuxia guest
// arrival. Copy discipline: warm endings only (U131, "never got to" sentence
// family banned), zero policy / mild-horror tokens (U129), zero stale meme
// verbatim, zero forced homophone puns (anti-cliche trio). A missing or corrupt
// data file degrades every Get(key) to the key itself (ASCII face, never tofu
// garbage); the font-coverage gate probes the same table the runtime renders.
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
namespace Sy.MonsterInn
{
    public static class LocalizeService
    {
        static Dictionary&lt;string, string&gt; _zh;
        static Dictionary&lt;string, string&gt; Table
        {
            get
            {
                if (_zh == null)
                {
                    var ta = Resources.Load&lt;TextAsset&gt;("Texts_zhCN");
                    _zh = ta == null ? new Dictionary&lt;string, string&gt;() : ParseTable(ta.text);
                }
                return _zh;
            }
        }
        public static int KeyCount { get { return Table.Count; } }
        /// &lt;summary&gt;Shipped zh display face (B757 census subset): the Resources stem
        /// of the forged NotoSansSC subset, no extension. Views resolve this name
        /// and degrade to the built-in face on a miss (U184 real-device CJK law -
        /// LegacyRuntime.ttf ships no zh glyphs).&lt;/summary&gt;
        public const string ShippedFontName = "NotoSansSC-G20Subset";
        /// &lt;summary&gt;Every table key: the font-coverage gate walks the whole
        /// player-visible value set, so new copy joins the probe with zero drift
        /// (single source, no parallel key list to maintain).&lt;/summary&gt;
        public static IEnumerable&lt;string&gt; AllKeys
        {
            get
            {
                foreach (var k in Table.Keys) yield return k;
            }
        }
        /// &lt;summary&gt;Missing key falls back to the key itself (gates read typos as diffs).&lt;/summary&gt;
        public static string Get(string key)
        {
```

```
            string v;
            return Table.TryGetValue(key, out v) ? v : key;
        }
        /// &lt;summary&gt;Parses a raw table payload (the data file face, shared with
        /// editor gates): non-empty string entries only, anything else drops the
        /// entry so lookups degrade to the key, never to garbage.&lt;/summary&gt;
        public static Dictionary&lt;string, string&gt; ParseTable(string json)
        {
            var map = new Dictionary&lt;string, string&gt;();
            if (string.IsNullOrEmpty(json)) return map;
            try
            {
                if (!(MiniJson.Deserialize(json) is Dictionary&lt;string, object&gt; d)) return map;
                foreach (var kv in d)
                    if (!string.IsNullOrEmpty(kv.Key) &amp;&amp; kv.Value is string s &amp;&amp; !string.IsNullOrEmpty(s
                        map[kv.Key] = s;
            }
            catch
            {
                // corrupt data file = empty table = key fallback face
                return new Dictionary&lt;string, string&gt;();
            }
            return map;
        }
    }
}
// SaveWire.cs
// SaveWire.cs - T009-a save face: one payload, eight facets.
// guests/records/wallet/codex/inn/stay/events/adcaps (task book S1 list, frozen).
// Capture() mirrors the six flow data faces into a plain dictionary; Restore() writes
// back field-wise through tolerant getters so unknown keys and missing facets never
// crash an older payload (v1-&gt;v2 migration face: absent keys keep live values).
// Storage goes through ISaveService (mock = PlayerPrefs; zero-server doctrine: local
// is the authority). MiniJson carries the wire because JsonUtility would drop unknown
// fields. Transient per-night phases (seat/serve phase machine, caption latch, event
// log streams) are session faces and intentionally NOT persisted - only durable
// counters and ledgers ride the save.
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.MonsterInn
{
    public static class SaveWire
    {
        public const string Slot = "monsterinn_save_v1";
        public const int FacetCount = 8;
        // ---- capture: live flows -&gt; payload ----
        public static Dictionary&lt;string, object&gt; Capture(InnFlow inn, ServeShift shift,
            CodexFlow codex, StayFlow stay, EventFlow events, AdFlow ads)
        {
```

```
var d = new Dictionary&lt;string, object&gt;();
// facet 1: guests (arrival counter drives the 12-roster determinism)
d["guests"] = new Dictionary&lt;string, object&gt; { { "arrivals", inn.Arrivals } };
// facet 2: records (service ledger + shift counters)
var recList = new List&lt;object&gt;();
for (int i = 0; i &lt; shift.Records.Count; i++)
{
    var r = shift.Records[i];
    recList.Add(new Dictionary&lt;string, object&gt;
    {
        { "arriveT", r.ArriveT }, { "settleT", r.SettleT }, { "species", r.Species },
        { "served", r.Served }, { "walkedOut", r.WalkedOut }, { "ops", r.Ops }
    });
}
d["records"] = new Dictionary&lt;string, object&gt;
{
    { "list", recList }, { "guestsStarted", shift.GuestsStarted }, { "served", shift.Served },
    { "walkOuts", shift.WalkOuts }, { "opsTotal", shift.OpsTotal }, { "opsMax", shift.OpsMax }
};
// facet 3: wallet (fortune-coin in/out accounting)
d["wallet"] = new Dictionary&lt;string, object&gt;
{
    { "coins", shift.Wallet.Coins }, { "coinsIn", shift.Wallet.CoinsIn },
    { "coinsOut", shift.Wallet.CoinsOut }
};
// facet 4: codex (first-seen unlocks)
var entries = new List&lt;object&gt;();
for (int i = 0; i &lt; codex.Entries.Count; i++)
    entries.Add(new Dictionary&lt;string, object&gt;
    {
        { "species", codex.Entries[i].Species }, { "unlockT", codex.Entries[i].UnlockT }
    });
d["codex"] = new Dictionary&lt;string, object&gt; { { "entries", entries } };
// facet 5: inn (growth spine + cozy rooms)
d["inn"] = new Dictionary&lt;string, object&gt;
{
    { "level", codex.Inn.Level }, { "cost", codex.Inn.Cost }, { "cozyRooms", codex.CozyRooms }
};
// facet 6: stay (X973 dual anchors + idempotency key + lifetime offline)
var lg = stay.Ledger;
d["stay"] = new Dictionary&lt;string, object&gt;
{
    { "stayers", lg.Stayers }, { "ratePerHour", lg.RatePerHour }, { "awayHours", lg.AwayHours },
    { "cap", lg.Cap }, { "wallSavedAt", lg.WallSavedAt }, { "uptimeSavedAt", lg.UptimeSavedAt },
    { "lastSettledNight", lg.LastSettledNight }, { "offlineEarned", lg.OfflineEarned }
};
// facet 7: events (comedy-family counters + night idempotency)
var fam = new List&lt;object&gt;();
for (int i = 0; i &lt; events.Ledger.FamilyCount.Length; i++) fam.Add(events.Ledger.FamilyCount[i]);
d["events"] = new Dictionary&lt;string, object&gt;
```

```
        {
            { "familyCount", fam }, { "lastEventNight", events.Ledger.LastEventNight },
            { "totalFired", events.Ledger.TotalFired }
        };
        // facet 8: adcaps (rolling-window capsules + lifetime totals)
        var caps = new List&lt;object&gt;();
        for (int i = 0; i &lt; ads.Ledger.Capsules.Count; i++)
        {
            var c = ads.Ledger.Capsules[i];
            caps.Add(new Dictionary&lt;string, object&gt;
            {
                { "type", c.Type }, { "offeredAt", c.OfferedAt }, { "accepted", c.Accepted }
            });
        }
        d["adcaps"] = new Dictionary&lt;string, object&gt;
        {
            { "capsules", caps }, { "offersTotal", ads.Ledger.OffersTotal },
            { "acceptsTotal", ads.Ledger.AcceptsTotal }
        };
        return d;
    }
    // ---- restore: payload -&gt; live flows (tolerant, never throws on absent data) ----
    public static void Restore(Dictionary&lt;string, object&gt; payload, InnFlow inn, ServeShift shift,
        CodexFlow codex, StayFlow stay, EventFlow events, AdFlow ads)
    {
        var g = Facet(payload, "guests");
        inn.Arrivals = GetInt(g, "arrivals", inn.Arrivals);
        var r = Facet(payload, "records");
        shift.Records.Clear();
        if (r.TryGetValue("list", out var rawList) &amp;&amp; rawList is List&lt;object&gt; rl)
            for (int i = 0; i &lt; rl.Count; i++)
            {
                var m = rl[i] as Dictionary&lt;string, object&gt;;
                if (m == null) continue;
                shift.Records.Add(new GuestRecord
                {
                    ArriveT = GetDouble(m, "arriveT", 0.0), SettleT = GetDouble(m, "settleT", 0.0),
                    Species = GetStr(m, "species", ""), Served = GetBool(m, "served", false),
                    WalkedOut = GetBool(m, "walkedOut", false), Ops = GetInt(m, "ops", 0)
                });
            }
        shift.GuestsStarted = GetInt(r, "guestsStarted", shift.GuestsStarted);
        shift.Served = GetInt(r, "served", shift.Served);
        shift.WalkOuts = GetInt(r, "walkOuts", shift.WalkOuts);
        shift.OpsTotal = GetInt(r, "opsTotal", shift.OpsTotal);
        shift.OpsMax = GetInt(r, "opsMax", shift.OpsMax);
        var w = Facet(payload, "wallet");
        shift.Wallet.Coins = GetDouble(w, "coins", shift.Wallet.Coins);
        shift.Wallet.CoinsIn = GetDouble(w, "coinsIn", shift.Wallet.CoinsIn);
        shift.Wallet.CoinsOut = GetDouble(w, "coinsOut", shift.Wallet.CoinsOut);
```

```
        var c = Facet(payload, "codex");
        codex.Entries.Clear();
        if (c.TryGetValue("entries", out var rawEntries) &amp;&amp; rawEntries is List&lt;object&gt; el)
            for (int i = 0; i &lt; el.Count; i++)
            {
                var m = el[i] as Dictionary&lt;string, object&gt;;
                if (m == null) continue;
                codex.Entries.Add(new CodexEntry
                {
                    Species = GetStr(m, "species", ""), UnlockT = GetDouble(m, "unlockT", 0.0)
                });
            }
        var n = Facet(payload, "inn");
        codex.Inn.Level = GetInt(n, "level", codex.Inn.Level);
        codex.Inn.Cost = GetDouble(n, "cost", codex.Inn.Cost);
        codex.CozyRooms = GetInt(n, "cozyRooms", codex.CozyRooms);
        var st = Facet(payload, "stay");
        stay.Ledger.Stayers = GetDouble(st, "stayers", stay.Ledger.Stayers);
        stay.Ledger.RatePerHour = GetDouble(st, "ratePerHour", stay.Ledger.RatePerHour);
        stay.Ledger.AwayHours = GetDouble(st, "awayHours", stay.Ledger.AwayHours);
        stay.Ledger.Cap = GetDouble(st, "cap", stay.Ledger.Cap);
        stay.Ledger.WallSavedAt = GetDouble(st, "wallSavedAt", stay.Ledger.WallSavedAt);
        stay.Ledger.UptimeSavedAt = GetDouble(st, "uptimeSavedAt", stay.Ledger.UptimeSavedAt);
        stay.Ledger.LastSettledNight = GetInt(st, "lastSettledNight", stay.Ledger.LastSettledNight);
        stay.Ledger.OfflineEarned = GetDouble(st, "offlineEarned", stay.Ledger.OfflineEarned);
        var ev = Facet(payload, "events");
        if (ev.TryGetValue("familyCount", out var rawFam) &amp;&amp; rawFam is List&lt;object&gt; fl)
            for (int i = 0; i &lt; fl.Count &amp;&amp; i &lt; events.Ledger.FamilyCount.Length; i++)
                events.Ledger.FamilyCount[i] = GetInt(new Dictionary&lt;string, object&gt;
                {
                    { "v", fl[i] }
                }, "v", events.Ledger.FamilyCount[i]);
        events.Ledger.LastEventNight = GetInt(ev, "lastEventNight", events.Ledger.LastEventNight);
        events.Ledger.TotalFired = GetInt(ev, "totalFired", events.Ledger.TotalFired);
        var ad = Facet(payload, "adcaps");
        ads.Ledger.Capsules.Clear();
        if (ad.TryGetValue("capsules", out var rawCaps) &amp;&amp; rawCaps is List&lt;object&gt; cl)
            for (int i = 0; i &lt; cl.Count; i++)
            {
                var m = cl[i] as Dictionary&lt;string, object&gt;;
                if (m == null) continue;
                ads.Ledger.Capsules.Add(new AdCapsule
                {
                    Type = GetStr(m, "type", ""), OfferedAt = GetDouble(m, "offeredAt", 0.0),
                    Accepted = GetBool(m, "accepted", false)
                });
            }
        ads.Ledger.OffersTotal = GetInt(ad, "offersTotal", ads.Ledger.OffersTotal);
        ads.Ledger.AcceptsTotal = GetInt(ad, "acceptsTotal", ads.Ledger.AcceptsTotal);
    }
```

```
        // ---- storage faces (through ISaveService; local is the authority) ----
        public static void Save(ISaveService save, InnFlow inn, ServeShift shift,
            CodexFlow codex, StayFlow stay, EventFlow events, AdFlow ads)
        {
            save.Save(Slot, MiniJson.Serialize(Capture(inn, shift, codex, stay, events, ads)));
        }
        public static Dictionary&lt;string, object&gt; Load(ISaveService save)
        {
            var json = save.Load(Slot);
            if (string.IsNullOrEmpty(json)) return null;
            return MiniJson.Deserialize(json) as Dictionary&lt;string, object&gt;;
        }
        // ---- wire codec faces (editor gates go through these: the Editor asmdef
        // intentionally references only the game runtime, never Sy.Core direct) ----
        public static string Encode(Dictionary&lt;string, object&gt; payload)
        {
            return MiniJson.Serialize(payload);
        }
        public static Dictionary&lt;string, object&gt; ThroughWire(Dictionary&lt;string, object&gt; payload)
        {
            return MiniJson.Deserialize(MiniJson.Serialize(payload)) as Dictionary&lt;string, object&gt;;
        }
        // ---- tolerant getters (MiniJson yields long/double/string/bool/null) ----
        static Dictionary&lt;string, object&gt; Facet(Dictionary&lt;string, object&gt; payload, string key)
        {
            if (payload == null) return new Dictionary&lt;string, object&gt;();
            return payload.TryGetValue(key, out var v) &amp;&amp; v is Dictionary&lt;string, object&gt; m
                ? m
                : new Dictionary&lt;string, object&gt;();
        }
        static double GetDouble(Dictionary&lt;string, object&gt; d, string key, double fallback)
        {
            return d.TryGetValue(key, out var v) &amp;&amp; v != null ? Convert.ToDouble(v) : fallback;
        }
        static int GetInt(Dictionary&lt;string, object&gt; d, string key, int fallback)
        {
            return d.TryGetValue(key, out var v) &amp;&amp; v != null ? Convert.ToInt32(v) : fallback;
        }
        static bool GetBool(Dictionary&lt;string, object&gt; d, string key, bool fallback)
        {
            return d.TryGetValue(key, out var v) &amp;&amp; v is bool b ? b : fallback;
        }
        static string GetStr(Dictionary&lt;string, object&gt; d, string key, string fallback)
        {
            return d.TryGetValue(key, out var v) &amp;&amp; v is string s ? s : fallback;
        }
    }
}
// ServeFlow.cs
// ServeFlow.cs - S1 service-ops + economy core (T004): ports the frozen E4 graybox
```

```
// M4/M5 face into the real runtime. Seat the guest (op1: tap a table), serve the dish
// (op2: a red-line-11 same-tick triple - serve tap + sfx + served-count pop all land
// on one tick), the meal runs its frozen 2.00s, then checkout credits lucky coins
// (+2.0 pop). Ops are capped at 2 per guest (E3#1 light-interaction idle discipline).
// A guest not served within the frozen 6.00s patience simply walks out - zero
// punishment (E3#5): no penalty events, no wallet loss, the next guest flows in
// untouched. The fortune wallet tracks in/out (S3) so the T005 upgrade slice can debit
// it. Pure C# (no UnityEngine) so Editor gates drive it headless; the view layer binds
// in T009. Every number is a frozen E4 anchor (GrayboxSim.cs) - zero re-tuning, zero
// RNG by design.
using System;
using System.Collections.Generic;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;One guest's service ledger line (S2 data face, E4 GuestRecord port).&lt;/summary&gt;
    public sealed class GuestRecord
    {
        public double ArriveT;
        public double SettleT;
        public string Species;
        public bool Served;
        public bool WalkedOut;
        public int Ops;
        public double Duration { get { return SettleT - ArriveT; } }
    }
    /// &lt;summary&gt;
    /// Fortune-coin wallet (S3 economy face): in/out accounting with an overdraft guard.
    /// Zero-punishment law: walkouts never touch the wallet, only settles credit it.
    /// &lt;/summary&gt;
    public sealed class FortuneWallet
    {
        public double Coins;
        public double CoinsIn;
        public double CoinsOut;
        public void AddCoins(double amount)
        {
            Coins += amount;
            CoinsIn += amount;
        }
        // Debit face (consumed by the T005 upgrade slice); refuses overdrafts with zero
        // mutation so a failed spend can never create negative coins.
        public bool SpendCoins(double amount)
        {
            if (amount &gt; Coins + 1e-9) return false;
            Coins -= amount;
            CoinsOut += amount;
            return true;
        }
    }
    /// &lt;summary&gt;
```

```
/// M4 seat + M5 serve machine for the nightly guest shift. The caller pumps Tick(dt)
/// on the shift clock; a served meal auto-settles at serve+2.00 (meal done + coin
/// pop + wallet credit) and an un-served guest auto-walks at arrive+6.00 (honest loss
/// face, zero punishment). One guest at a time - the E4 bot drives a sequential
/// shift, so a new guest only starts once the previous one settled or walked out.
/// &lt;/summary&gt;
public sealed class ServeShift
{
    private enum Phase { None, Waiting, Seated, Served, Settled, WalkedOut }
    public double Time;   // shift-local logical seconds
    public readonly List&lt;InnEvent&gt; Events = new List&lt;InnEvent&gt;();
    public readonly List&lt;GuestRecord&gt; Records = new List&lt;GuestRecord&gt;();
    public readonly FortuneWallet Wallet = new FortuneWallet();
    public int GuestsStarted, Served, WalkOuts, OpsTotal, OpsMax;
    private Phase _phase = Phase.None;
    private GuestSpec _guest;
    private GuestRecord _record;
    private double _arriveT;
    private double _serveT;
    public bool GuestInFlight
    {
        get { return _phase == Phase.Waiting || _phase == Phase.Seated || _phase == Phase.Served; }
    }
    public void Log(string type, string detail) { Events.Add(new InnEvent(Time, type, detail)); }
    public double TimeOf(string type)
    {
        for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) return Events[i].T;
        return -1.0;
    }
    public int CountEvents(string type)
    {
        int n = 0;
        for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) n++;
        return n;
    }
    // Starts one guest's service window (the reveal itself already happened in
    // InnFlow). Sequential shift: rejects while a previous guest is still in flight.
    public bool BeginGuest(GuestSpec guest)
    {
        if (guest == null || GuestInFlight) return false;
        _guest = guest;
        _arriveT = Time;
        _record = new GuestRecord { ArriveT = Time, Species = guest.Species };
        _phase = Phase.Waiting;
        GuestsStarted++;
        return true;
    }
    // op1: the player taps a table to seat the waiting guest.
    public bool Seat()
    {
```

```
    if (_phase != Phase.Waiting) return false;
    _phase = Phase.Seated;
    Log("seat_tap", "op1: tap a table to seat the guest");
    CountOp();
    InnAudioHooks.OnSeat();   // T010: same-tick seat hook (real success path)
    return true;
}
// op2: the signature dish lands - red-line-11 same-tick triple: serve tap + sfx
// + served-count pop all fire on this one tick with zero time between them.
public bool Serve()
{
    if (_phase != Phase.Seated) return false;
    _phase = Phase.Served;
    _serveT = Time;
    Log("serve_tap", "op2: the signature dish lands steaming");
    Log("sfx_slurp", "happy slurp");
    Log("served_count", "served=" + (Served + 1).ToString("0", System.Globalization.CultureInfo.InvariantCulture
    CountOp();
    InnAudioHooks.OnServe();  // T010: same-tick serve hook (triple audio face)
    return true;
}
private void CountOp()
{
    OpsTotal++;
    _record.Ops++;
    if (_record.Ops &gt; OpsMax) OpsMax = _record.Ops;
}
// Advance the shift clock. Terminal phases are inert (no double settle, no
// post-walkout events - idempotency by phase).
public void Tick(double dt)
{
    Time += dt;
    if (_phase == Phase.Served &amp;&amp; Time + 1e-9 &gt;= _serveT + InnFrozen.MealDur)
    {
        FinishMeal();
    }
    else if ((_phase == Phase.Waiting || _phase == Phase.Seated)
        &amp;&amp; Time + 1e-9 &gt;= _arriveT + InnFrozen.WalkPatience)
    {
        GuestWalksOut();
    }
}
// Checkout: the meal settles - frozen detail strings, wallet credit, ledger line.
private void FinishMeal()
{
    _phase = Phase.Settled;
    Log("meal_done", "the guest pats its round belly");
    Log("coin_pop", "+" + _guest.Pay.ToString("0", System.Globalization.CultureInfo.InvariantCulture) + " lucky
    Wallet.AddCoins(_guest.Pay);
    InnAudioHooks.OnSettle(); // T010: same-tick checkout hook (coin pop face)
```

```
            _record.SettleT = Time;
            _record.Served = true;
            Served++;
            Records.Add(_record);
        }
        // Honest loss face (E3#5 zero punishment): the un-served guest simply walks
        // out - no penalty events, no wallet loss, nothing blocks the next guest.
        private void GuestWalksOut()
        {
            _phase = Phase.WalkedOut;
            Log("guest_walks_out", _guest.Species + " yawns and drifts back into the night");
            InnAudioHooks.OnWalkout();  // T010: same-tick walkout puff hook (honest loss, zero punishment)
            _record.SettleT = Time;
            _record.WalkedOut = true;
            WalkOuts++;
            Records.Add(_record);
        }
    }
}
// ShareCodec.cs
// ShareCodec.cs - S1 share-codec core (T007): ports the frozen E4 graybox M10 face
// into the real runtime. The share card shows off the yokai codex lineup as an
// X032-style deterministic code: the "MI1-" prefix followed by one uppercase hex
// digit per unlocked roster species (the roster index), so a full 12-codex composes
// "MI1-0123456789AB" (12 hex digits = the E4 measured 12) and an 8-species lineup
// already clears the &gt;= 8-hex floor (north-star vii). Decode is the pure visitor
// face: a well-formed code restores the exact species lineup in roster order, and
// any malformed input (null, empty, prefix-only, wrong prefix, lowercase, non-hex
// char, out-of-roster digit, duplicated digit) heals to null - zero exception,
// zero partial restore, zero state mutation. Purely local (red line 12: zero
// server, zero network). Pure C# (no UnityEngine) so Editor gates drive it
// headless; the view layer binds in T009. Zero RNG by design.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;
    /// M10 compose/decode pair (S8 API face): codex lineup &lt;-&gt; "MI1-" hex code.
    /// Both directions are pure functions - the codec never owns state.
    /// &lt;/summary&gt;
    public static class ShareCodec
    {
        public const string Prefix = "MI1-";
        // Compose the lineup code: one uppercase hex digit per unlocked roster
        // species, walked in frozen roster order (deterministic for a given set).
        public static string ComposeShare(IEnumerable&lt;string&gt; unlockedSpecies)
        {
            var sb = new StringBuilder(Prefix);
```

```
            if (unlockedSpecies != null)
            {
                for (int i = 0; i &lt; InnFrozen.CodexRoster.Length; i++)
                {
                    bool has = false;
                    foreach (string s in unlockedSpecies)
                        if (s == InnFrozen.CodexRoster[i]) { has = true; break; }
                    if (has) sb.Append(i.ToString("X", CultureInfo.InvariantCulture));
                }
            }
            return sb.ToString();
        }
        // Visitor decode face: a well-formed code restores the exact species lineup
        // in roster order. Any malformed input heals to null - the codec never
        // crashes a visitor and never half-restores a lineup. An empty payload is
        // malformed (a share card always carries at least one yokai).
        public static List&lt;string&gt; DecodeShare(string code)
        {
            if (code == null) return null;
            if (code.Length &lt;= Prefix.Length) return null;
            if (!code.StartsWith(Prefix, StringComparison.Ordinal)) return null;
            var lineup = new List&lt;string&gt;();
            int seen = 0;
            for (int k = Prefix.Length; k &lt; code.Length; k++)
            {
                int index = HexDigit(code[k]);
                if (index &lt; 0 || index &gt;= InnFrozen.RosterSize) return null; // non-hex or off-roster
                int bit = 1 &lt;&lt; index;
                if ((seen &amp; bit) != 0) return null;                          // duplicate digit
                seen |= bit;
                lineup.Add(InnFrozen.CodexRoster[index]);
            }
            return lineup;
        }
        // Uppercase hex only - the composer is the frozen spelling authority.
        static int HexDigit(char c)
        {
            if (c &gt;= '0' &amp;&amp; c &lt;= '9') return c - '0';
            if (c &gt;= 'A' &amp;&amp; c &lt;= 'F') return c - 'A' + 10;
            return -1;
        }
    }
    /// &lt;summary&gt;
    /// S8 share-card machine: the player-tap face composes the card at any moment
    /// (ComposeCard), and TryShareCard reproduces the frozen E4 bot cadence for the
    /// gate drive (the lineup is worth showing at codex &gt;= 8 on nights n==2 or
    /// n%5==0 - 4 cards across the 20-night run). Purely local: compose + log, zero
    /// server, zero network (red line 12).
    /// &lt;/summary&gt;
    public sealed class ShareFlow
```

```
    {
        public double Time;   // night-clock logical seconds (view binds in T009)
        public readonly List&lt;InnEvent&gt; Events = new List&lt;InnEvent&gt;();
        public int ShareCards;
        public void Log(string type, string detail) { Events.Add(new InnEvent(Time, type, detail)); }
        public int CountEvents(string type)
        {
            int n = 0;
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) n++;
            return n;
        }
        // Advance the night clock (the share beat is caller-driven, not self-firing).
        public void Tick(double dt) { Time += dt; }
        // Frozen E4 bot cadence: the card fires once the codex lineup is worth
        // showing (&gt;= 8 unlocked) on the cadence nights (n == 2, then n % 5 == 0).
        public string TryShareCard(int nightIndex, CodexFlow codex)
        {
            if (codex == null || codex.CountUnlocked &lt; 8) return null;
            if (nightIndex != 2 &amp;&amp; (nightIndex &lt;= 2 || nightIndex % 5 != 0)) return null;
            return ComposeCard(codex);
        }
        // Player-tap face (S03_Share binds in T009): compose + log the card.
        public string ComposeCard(CodexFlow codex)
        {
            string code = ShareCodec.ComposeShare(CodexSpeciesOf(codex));
            ShareCards++;
            Log("share_card", "code=" + code + " (show off your yokai codex)");
            return code;
        }
        static IEnumerable&lt;string&gt; CodexSpeciesOf(CodexFlow codex)
        {
            if (codex == null) yield break;
            for (int i = 0; i &lt; codex.Entries.Count; i++) yield return codex.Entries[i].Species;
        }
    }
}
// StayFlow.cs
// StayFlow.cs - S1 night-stay offline settlement core (T006): ports the frozen E4
// graybox M8 face into the real runtime. Morning settlement of overnight stayers:
// away earnings = stayers x rate x the frozen 8.0h away window, capped per night
// (base(2)=16, five stayers cap at 20). The X973 dual-anchor wall-clock domain guards
// the offline face: the save ledger carries WallSavedAt + UptimeSavedAt, a resume with
// a rolled-back wall clock clamps the away window to zero (zero pay, zero double
// pay), and a fresh ledger without anchors never grants free coins. A settled night
// never pays twice: the last-settled-night key makes every settlement idempotent no
// matter which clock face calls it, and the offline gate reuses the same settle core
// so there is exactly one settlement path (zero future drift). The wallet is
// injected per call (ServeShift.Wallet stays the single economy source). Pure C#
// (no UnityEngine) so Editor gates drive it headless; the view layer binds in T009.
// Every number is a frozen E4 anchor (GrayboxSim.cs) - zero re-tuning, zero RNG.
```

```
using System;
using System.Collections.Generic;
namespace Sy.MonsterInn
{
    /// &lt;summary&gt;Night-stay ledger (S6 data face, save.stay) with the X973 dual anchors.&lt;/summary&gt;
    public sealed class StayLedger
    {
        public double Stayers;                                  // last settled night's stayer count
        public double RatePerHour = InnFrozen.StayRatePerHour;
        public double AwayHours = InnFrozen.AwayHours;
        public double Cap = InnFrozen.StayCapPerNight;          // S1 base cap face (persisted; live ceiling derives from
        public double WallSavedAt;                              // X973 wall anchor (wall seconds at save)
        public double UptimeSavedAt;                             // X973 session anchor (uptime seconds at save)
        public int LastSettledNight = -1;                        // idempotency key: settled nights never repay
        public double OfflineEarned;                             // lifetime offline accumulation
    }
    /// &lt;summary&gt;
    /// M8 night-stay offline settlement machine. The caller stamps the dual anchors at
    /// save time, pumps Tick(dt) on the settlement clock, and fires MorningSettlement on
    /// the morning beat (in-session nights) or SettleOfflineNight after a resume.
    /// &lt;/summary&gt;
    public sealed class StayFlow
    {
        public double Time;   // settlement-local logical seconds
        public readonly List&lt;InnEvent&gt; Events = new List&lt;InnEvent&gt;();
        public readonly StayLedger Ledger = new StayLedger();
        public void Log(string type, string detail) { Events.Add(new InnEvent(Time, type, detail)); }
        public double TimeOf(string type)
        {
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) return Events[i].T;
            return -1.0;
        }
        public int CountEvents(string type)
        {
            int n = 0;
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].Type == type) n++;
            return n;
        }
        // Advance the settlement clock (the morning beat is caller-driven, not self-firing).
        public void Tick(double dt) { Time += dt; }
        // X973 dual-anchor write: stamp both clock faces at save time.
        public void SaveAnchors(double wallNow, double uptimeNow)
        {
            Ledger.WallSavedAt = wallNow;
            Ledger.UptimeSavedAt = uptimeNow;
        }
        // X973 resume face: away seconds since the save anchor. A fresh ledger (no
        // anchor yet) pays nothing, and a rolled-back wall clock clamps to zero - both
        // faces are zero free coins, never negative, never a double pay.
        public double ResumeAwaySeconds(double wallNow)
```

```
        {
            if (Ledger.WallSavedAt &lt;= 0.0) return 0.0;           // fresh face: zero free pay
            return Math.Max(0.0, wallNow - Ledger.WallSavedAt);  // reverse clock: away = 0
        }
        // M8 morning settlement (S6 API face, E4 frozen): one idempotent settle per
        // night index. The stay ceiling grows with the inn level (S2 slope, T012-c):
        // cap(L) = 20 + 4*(L-1), so cozy rooms raise what one night can bank. An
        // already-settled night (or any older one) is a silent no-op: zero wallet
        // mutation, zero events - the wall-clock domain can never double-pay.
        public double MorningSettlement(int stayers, int nightIndex, int innLevel, FortuneWallet wallet)
        {
            if (nightIndex &lt;= Ledger.LastSettledNight) return 0.0;
            double raw = stayers * Ledger.RatePerHour * Ledger.AwayHours;
            double settled = Math.Min(raw, InnFrozen.StayCapFor(innLevel));
            Ledger.LastSettledNight = nightIndex;
            Ledger.Stayers = stayers;
            Ledger.OfflineEarned += settled;
            if (wallet != null) wallet.AddCoins(settled);
            Log("morning_settlement", "overnight stay earnings: "
                + settled.ToString("0", System.Globalization.CultureInfo.InvariantCulture)
                + " coins (stayers=" + stayers.ToString(System.Globalization.CultureInfo.InvariantCulture) + ")");
            InnAudioHooks.OnMorningSettle();  // T010: same-tick morning settle jingle hook
            return settled;
        }
        // Offline resume settle (X973 single settle path): the morning settlement runs
        // the same core, gated by the away window. No away time (fresh face, rolled-back
        // clock, or the same instant) = zero pay with zero mutation, and the night stays
        // unsettled so a later legit window still pays it exactly once.
        public double SettleOfflineNight(int stayers, int nightIndex, int innLevel, double wallNow, FortuneWallet wallet
        {
            if (ResumeAwaySeconds(wallNow) &lt;= 0.0) return 0.0;
            return MorningSettlement(stayers, nightIndex, innLevel, wallet);
        }
    }
}
```